

# Phase 5 — Computer Vision (CRITICAL)

---

## ExamGuard AI — Implementation Guide

### OpenCV — The Foundation of Computer Vision

#### What Is This?

OpenCV (Open Source Computer Vision Library) is THE Python library for working with images and video. If you want to read a camera feed, draw a box around a detected face, resize an image, or convert colors — you use OpenCV.

Think of it this way:

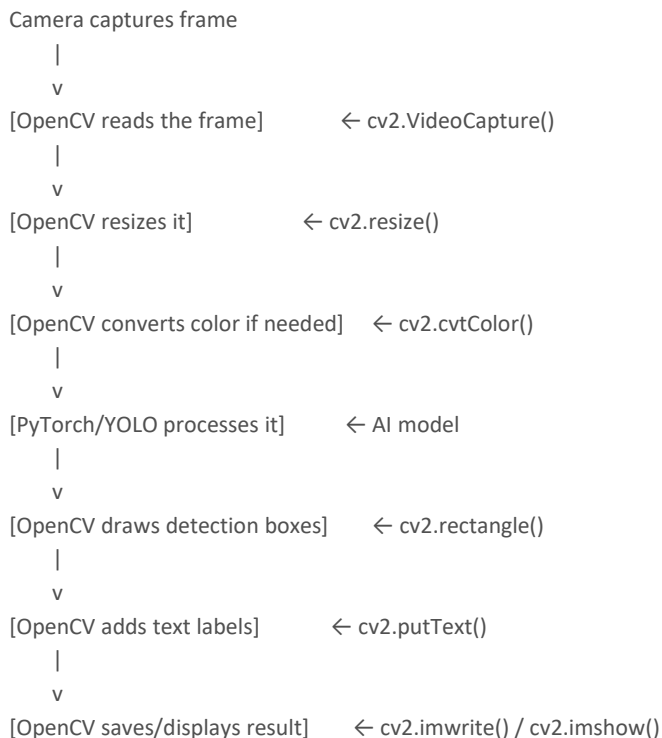
- PyTorch = the brain (makes decisions about what it sees)
- OpenCV = the eyes and hands (reads images, draws results, processes frames)

Every computer vision project in the world uses OpenCV. It has been around since 2000, it is free, and it works on every platform.

#### WHY OpenCV Is Essential for ExamGuard

Every single camera frame passes through OpenCV before any AI model ever sees it.

Here is the ExamGuard frame processing pipeline:



Without OpenCV, you cannot:

- Read camera feeds

- Process video frames
- Draw bounding boxes around detected cheaters
- Save evidence clips
- Display the monitoring dashboard

OpenCV is like learning to drive before you can race. You need it for everything.

## What You Need to Learn

### 1. Reading Images

```
import cv2
```

```
# Read an image from file
```

```
image = cv2.imread('exam_frame.jpg')
```

```
# Check what you got
```

```
print(type(image))    # <class 'numpy.ndarray'>
```

```
print(image.shape)    # (1080, 1920, 3) → height, width, 3 color channels
```

```
print(image.dtype)    # uint8 → pixel values 0-255
```

```
# Display the image
```

```
cv2.imshow('Exam Frame', image)
```

```
cv2.waitKey(0)        # Wait for any key press
```

```
cv2.destroyAllWindows()
```

ExamGuard connection: Every saved evidence frame is loaded this way for review.

### 2. Reading Video (Camera Feed)

```
import cv2
```

```
# Read from webcam (0 = default camera)
```

```
cap = cv2.VideoCapture(0)
```

```
# Read from video file
```

```
# cap = cv2.VideoCapture('exam_recording.mp4')
```

```
# Read from network camera (RTSP stream)
```

```
# cap = cv2.VideoCapture('rtsp://192.168.1.100:554/stream')
```

```
while True:
```

```
    ret, frame = cap.read()  # ret = success?, frame = the image
```

```
    if not ret:
```

```
        break
```

```
    # Display the frame
```

```
    cv2.imshow('Live Feed', frame)
```

```
    # Press 'q' to quit
```

```
    if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
        break
```

```
cap.release()
cv2.destroyAllWindows()
```

ExamGuard connection: This is literally how ExamGuard reads each exam hall camera. Replace `0` with an RTSP URL and you are reading a real security camera.

### 3. Drawing Rectangles (Bounding Boxes)

```
import cv2
```

```
image = cv2.imread('exam_frame.jpg')
```

```
# Draw a red rectangle around a detected student
# Parameters: image, top-left corner, bottom-right corner, color (BGR), thickness
cv2.rectangle(image, (100, 50), (300, 400), (0, 0, 255), 2)
```

```
# Draw a green rectangle for "normal" student
cv2.rectangle(image, (400, 50), (600, 400), (0, 255, 0), 2)
```

```
cv2.imshow('Detections', image)
cv2.waitKey(0)
```

ExamGuard connection: Every detected cheater gets a red box. Every normal student gets a green box. Every detected phone/chit gets a yellow box with label.

### 4. Adding Text Labels

```
import cv2
```

```
image = cv2.imread('exam_frame.jpg')
```

```
# Add text above the bounding box
cv2.putText(image, 'CHEATING 87%', (100, 45),
            cv2.FONT_HERSHEY_SIMPLEX, # Font
            0.8,                      # Font size
            (0, 0, 255),               # Red color (BGR)
            2)                        # Thickness
```

```
cv2.putText(image, 'Normal', (400, 45),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.8,
            (0, 255, 0),               # Green color
            2)
```

```
cv2.imshow('Labeled', image)
cv2.waitKey(0)
```

### 5. Resizing Images

```
import cv2
```

```
image = cv2.imread('exam_frame.jpg') # Original: 1920x1080
```

```
# Resize to specific dimensions (what AI models need)
```

```
resized = cv2.resize(image, (224, 224))
```

```
# Resize by percentage
```

```
half = cv2.resize(image, None, fx=0.5, fy=0.5) # 50% of original
```

```
print(f"Original: {image.shape}") # (1080, 1920, 3)
```

```
print(f"Resized: {resized.shape}") # (224, 224, 3)
```

```
print(f"Half: {half.shape}") # (540, 960, 3)
```

ExamGuard connection: Camera captures at 1920x1080, but CNN needs 224x224. OpenCV does the resizing.

## 6. Color Conversion

```
import cv2
```

```
image = cv2.imread('exam_frame.jpg') # Loaded in BGR (OpenCV default)
```

```
# Convert to RGB (what PyTorch/matplotlib expect)
```

```
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
# Convert to grayscale (for some models, faster processing)
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# Convert to HSV (for color-based detection)
```

```
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
```

Important: OpenCV loads images in BGR (Blue, Green, Red), not RGB. This trips up every beginner. Always convert to RGB before passing to PyTorch.

## 7. Saving Frames (Evidence)

```
import cv2
```

```
from datetime import datetime
```

```
image = cv2.imread('exam_frame.jpg')
```

```
# Save a single frame
```

```
cv2.imwrite('evidence_frame.jpg', image)
```

```
# Save with timestamp (for ExamGuard evidence logs)
```

```
timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
```

```
filename = f'evidence/cheating_{timestamp}.jpg'
```

```
cv2.imwrite(filename, image)
```

```
# Saves as: evidence/cheating_20260319_102345.jpg
```

ExamGuard connection: When cheating is detected, the system saves the frame with timestamp as evidence.

## ExamGuard OpenCV Pipeline — Complete Example

```
import cv2
```

```
import numpy as np
```

```
from datetime import datetime
```

```

# Read from exam hall camera
cap = cv2.VideoCapture('rtsp://192.168.1.100:554/stream')

# Get video properties
fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
print(f"Camera: {width}x{height} at {fps} FPS")

frame_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1

    # Skip frames for performance (process every 3rd frame)
    if frame_count % 3 != 0:
        continue

    # Resize for AI model
    model_input = cv2.resize(frame, (224, 224))
    model_input_rgb = cv2.cvtColor(model_input, cv2.COLOR_BGR2RGB)

    # --- HERE IS WHERE YOU PLUG IN YOUR AI MODEL ---
    # prediction = model.predict(model_input_rgb)
    # For now, simulate a detection:
    is_cheating = frame_count % 100 < 10 # Fake detection every 100 frames

    if is_cheating:
        # Draw red box and label
        cv2.rectangle(frame, (200, 100), (500, 450), (0, 0, 255), 3)
        cv2.putText(frame, 'CHEATING DETECTED', (200, 90),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

        # Save evidence
        timestamp = datetime.now().strftime('%Y-%m-%d_%H%M%S')
        cv2.imwrite(f'evidence/cheating_{timestamp}.jpg', frame)

    # Add timestamp overlay
    time_text = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    cv2.putText(frame, time_text, (10, height - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1)

    # Display
    cv2.imshow('ExamGuard - Hall A', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

```

```
cap.release()
cv2.destroyAllWindows()
```

## Mini Project: Motion Detection in Video

Goal: Read a video file and detect frames where something moves. This is the simplest form of surveillance and teaches you the core OpenCV skills.

```
import cv2
import numpy as np

# Read video (use webcam with 0, or a video file)
cap = cv2.VideoCapture(0)

# Read first frame as the "background"
ret, prev_frame = cap.read()
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
prev_gray = cv2.GaussianBlur(prev_gray, (21, 21), 0)

motion_detected_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Convert current frame to grayscale and blur
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray = cv2.GaussianBlur(gray, (21, 21), 0)

    # Calculate difference between current and previous frame
    diff = cv2.absdiff(prev_gray, gray)

    # Threshold: if difference > 25, it is motion
    thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)[1]

    # Dilate to fill gaps
    thresh = cv2.dilate(thresh, None, iterations=2)

    # Find contours (moving regions)
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)

    motion_in_frame = False

    for contour in contours:
        # Ignore tiny movements (noise)
        if cv2.contourArea(contour) < 5000:
            continue

    motion_in_frame = True
```

```

# Draw box around motion
x, y, w, h = cv2.boundingRect(contour)
cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

if motion_in_frame:
    motion_detected_count += 1
    cv2.putText(frame, f'MOTION DETECTED ({motion_detected_count})',
                (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

# Show result
cv2.imshow('Motion Detection', frame)
cv2.imshow('Threshold', thresh)

# Update previous frame
prev_gray = gray.copy()

if cv2.waitKey(30) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
print(f"Total frames with motion: {motion_detected_count}")

```

### What You Will Learn:

- Reading video frame by frame
- Converting between color and grayscale
- Image subtraction (finding differences between frames)
- Thresholding (converting gray differences to black/white)
- Finding and drawing contours
- Real-time display with OpenCV

### Connection to ExamGuard:

This is a primitive version of what ExamGuard does. Instead of just detecting "motion," ExamGuard uses AI to determine "that motion is a student passing a chit." But the OpenCV frame reading, processing, and display pipeline is identical.

### Key OpenCV Functions for ExamGuard

| Function             | What It Does     | ExamGuard Use                    |
|----------------------|------------------|----------------------------------|
| `cv2.VideoCapture()` | Read camera feed | Read exam hall cameras           |
| `cv2.imread()`       | Read image file  | Load saved evidence              |
| `cv2.resize()`       | Resize image     | Prep frames for AI model         |
| `cv2.cvtColor()`     | Convert colors   | BGR to RGB for PyTorch           |
| `cv2.rectangle()`    | Draw box         | Mark detected cheaters           |
| `cv2.putText()`      | Add text         | Label detections with confidence |
| `cv2.imwrite()`      | Save image       | Save evidence frames             |
| `cv2.imshow()`       | Display image    | Monitoring dashboard             |
| `cv2.GaussianBlur()` | Blur image       | Reduce noise before processing   |

## Key Takeaway

OpenCV is the first thing you use and the last thing you use in any computer vision pipeline. It reads the camera, it preprocesses for the AI model, it draws the results, and it saves the evidence. You cannot build ExamGuard without it. Learn these basics well — you will use them in every single lesson from here on.



## YOLO — Real-Time Object Detection for ExamGuard

### What Is This?

YOLO stands for You Only Look Once. It is an object detection model that can look at an image and instantly find all the objects in it — drawing boxes around each one with a label and confidence percentage.

Unlike image classification (which says "this image contains a phone"), object detection says "there is a phone at position (x=340, y=220), size 50x80 pixels, confidence 94%."

YOLO is special because it is fast enough for real-time video — it can process 30+ frames per second. Most other object detection models are too slow for live camera feeds.

### WHY YOLO Is THE Model for ExamGuard

ExamGuard needs to detect specific objects in exam halls in real time:

Camera frame from Exam Hall A

|  
v  
YOLO  
|  
v

Detected objects:

- Phone (94% confidence) at Row 3, Seat 5
- Chit/paper slip (82% confidence) at Row 2, Seat 8
- Earpiece (71% confidence) at Row 5, Seat 3
- Person (99% confidence) x 30 students

Without YOLO, ExamGuard cannot tell you WHAT was detected or WHERE it is. YOLO gives both.

### What ExamGuard Uses YOLO to Detect:

| Object           | Why It Matters                 | Confidence Threshold |
|------------------|--------------------------------|----------------------|
| Mobile phone     | #1 cheating tool               | 85%+                 |
| Paper chit/slip  | Passing answers                | 75%+                 |
| Earpiece/earbud  | Receiving answers from outside | 70%+                 |
| Smartwatch       | Can display answers            | 80%+                 |
| Extra paper      | Unauthorized notes             | 70%+                 |
| Person (student) | Track who is where             | 90%+                 |

### How YOLO Works

#### The Key Insight: Look Once

Older models (like R-CNN) look at an image thousands of times, checking different regions one by one. YOLO looks at the entire image once and predicts all objects simultaneously.

Old approach (R-CNN):

Image → Check region 1 → Check region 2 → ... → Check region 2000  
Result: 5-7 FPS (too slow for real-time)

YOLO approach:

Image → Look at everything at once → All detections  
Result: 30-100+ FPS (real-time!)

## Step by Step:

1. DIVIDE: Split image into a grid (e.g., 13x13)
2. PREDICT: Each grid cell predicts:
  - Bounding boxes (where are objects?)
  - Confidence scores (how sure are we?)
  - Class probabilities (what is it?)
3. FILTER: Remove low-confidence predictions
4. NMS (Non-Max Suppression): If multiple boxes overlap on the same object, keep only the best one
5. OUTPUT: Final list of detected objects with positions and labels

## Visual Example:

Input frame:

```
+---+---+---+---+---+
|   |   |   |   |   |
+---+---+---+---+---+
|   | P |   |   |   | P = Phone detected here
+---+---+---+---+---+
|   |   | C |   |   | C = Chit detected here
+---+---+---+---+---+
```

Output:

Object 1: "phone", confidence: 94%, box: (120, 80, 170, 160)  
Object 2: "chit", confidence: 82%, box: (290, 180, 340, 220)

## YOLO Versions — Which to Use

| Version | Year | Speed | Accuracy  | Recommendation                   |
|---------|------|-------|-----------|----------------------------------|
| YOLOv5  | 2020 | Fast  | Great     | Good, well-documented            |
| YOLOv7  | 2022 | Fast  | Better    | Advanced users                   |
| YOLOv8  | 2023 | Fast  | Best      | USE THIS — latest and easiest    |
| YOLOv9  | 2024 | Fast  | Excellent | Newest, less tutorials available |

Use YOLOv8 (by Ultralytics). It is the easiest to use, has the best documentation, and is actively maintained.

```
# Install YOLOv8
pip install ultralytics
```

## Using Pre-Trained YOLO (Out of the Box)

YOLOv8 comes pre-trained on the COCO dataset — 80 common objects including person, phone, book, etc.

```
from ultralytics import YOLO
import cv2
```

```
# Load pre-trained YOLOv8 model
```

```

model = YOLO('yolov8n.pt') # 'n' = nano (smallest, fastest)
                        # Options: yolov8n, yolov8s, yolov8m, yolov8l, yolov8x

# Detect objects in an image
results = model('exam_frame.jpg')

# Print detections
for result in results:
    for box in result.boxes:
        class_name = result.names[int(box.cls)]
        confidence = float(box.conf)
        x1, y1, x2, y2 = map(int, box.xyxy[0])

        print(f"Detected: {class_name} ({confidence:.1%}) at ({x1},{y1})-({x2},{y2})")

# Example output:
# Detected: person (97.3%) at (50,20)-(200,450)
# Detected: cell phone (89.1%) at (340,220)-(390,300)
# Detected: person (96.8%) at (250,20)-(400,450)

# Save image with drawn boxes
results[0].save('exam_frame_detected.jpg')

```

That is it. Five lines of code to detect objects in an image.

## YOLO on Live Video (Real-Time)

```

from ultralytics import YOLO
import cv2

model = YOLO('yolov8n.pt')

# Open webcam (or RTSP stream for real cameras)
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Run YOLO on this frame
    results = model(frame, verbose=False)

    # Draw results on frame
    annotated_frame = results[0].plot()

    # Display
    cv2.imshow('YOLO Detection', annotated_frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

```

```
cap.release()
cv2.destroyAllWindows()
```

That is real-time object detection in 15 lines of code. This is the power of YOLO.

## Training YOLO on Custom Data (ExamGuard Objects)

Pre-trained YOLO knows "cell phone" and "person" but does NOT know "chit," "earpiece," or "cheating behavior." You need to train it on your own data.

### Step 1: Collect and Label Data

You need images of exam-specific objects with bounding box annotations.

Tools for labeling:

- Labellmg (desktop app, free)
- Roboflow (web app, free tier available)
- CVAT (web app, free, open source)

For each image, you draw boxes and label them:

Image: exam\_frame\_001.jpg  
Box 1: "phone" at (340, 220, 390, 300)  
Box 2: "chit" at (150, 180, 200, 210)  
Box 3: "student" at (50, 20, 200, 450)

### Step 2: Organize Data

```
dataset/
train/
  images/
    frame_001.jpg
    frame_002.jpg
    ...
  labels/
    frame_001.txt (YOLO format: class_id x_center y_center width height)
    frame_002.txt
val/
  images/
  labels/
```

```
data.yaml:
train: dataset/train/images
val: dataset/val/images
nc: 5 # number of classes
names: ['student', 'phone', 'chit', 'earpiece', 'smartwatch']
```

### Step 3: Train

```
from ultralytics import YOLO
```

```
# Start from pre-trained model (transfer learning!)
model = YOLO('yolov8n.pt')
```

```
# Train on your custom dataset
results = model.train(
    data='data.yaml',
```

```

    epochs=50,
    imgsz=640,
    batch=16,
    name='examguard_detector'
)

```

# That is it! The model now detects your custom objects.

#### Step 4: Use Your Trained Model

```

# Load your custom-trained model
model = YOLO('runs/detect/examguard_detector/weights/best.pt')

```

```

# Detect exam-specific objects
results = model('exam_frame.jpg')

```

```

# Output:
# Detected: phone (94.2%) at (340,220)-(390,300)
# Detected: chit (82.5%) at (150,180)-(200,210)
# Detected: earpiece (71.3%) at (480,50)-(510,80)

```

#### Why YOLO Over Other Object Detection Models?

| Model              | FPS     | Use Case             | ExamGuard Verdict         |
|--------------------|---------|----------------------|---------------------------|
| YOLO               | 30-100+ | Real-time detection  | USE THIS                  |
| Faster R-CNN       | 5-7     | Accurate but slow    | Too slow for live feeds   |
| SSD                | 20-40   | Mobile devices       | Decent but YOLO is better |
| RetinaNet          | 5-10    | High accuracy needed | Too slow for 4+ cameras   |
| DETR (Transformer) | 3-5     | Research             | Way too slow              |

For ExamGuard processing 4 cameras at 30 FPS = 120 frames/second, only YOLO is fast enough.

#### ExamGuard YOLO Pipeline

```

from ultralytics import YOLO
import cv2
from datetime import datetime

# Load custom-trained ExamGuard model
model = YOLO('examguard_detector.pt')

# Connect to exam hall camera
cap = cv2.VideoCapture('rtsp://192.168.1.100:554/stream')

# Define alert thresholds
ALERT_THRESHOLDS = {
    'phone': 0.85,
    'chit': 0.75,
    'earpiece': 0.70,
    'smartwatch': 0.80
}

while True:
    ret, frame = cap.read()
    if not ret:

```

```

        break

# Run detection
results = model(frame, verbose=False)

for result in results:
    for box in result.bboxes:
        class_name = result.names[int(box.cls)]
        confidence = float(box.conf)
        x1, y1, x2, y2 = map(int, box.xyxy[0])

        # Check if this detection should trigger an alert
        if class_name in ALERT_THRESHOLDS:
            if confidence >= ALERT_THRESHOLDS[class_name]:
                # ALERT!
                print(f"ALERT: {class_name} detected ({confidence:.1%})")

                # Draw red box
                cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 0, 255), 3)
                cv2.putText(frame, f'{class_name} {confidence:.0%}',
                            (x1, y1-10), cv2.FONT_HERSHEY_SIMPLEX,
                            0.8, (0, 0, 255), 2)

                # Save evidence
                timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
                cv2.imwrite(f'evidence/{class_name}_{timestamp}.jpg', frame)

cv2.imshow('ExamGuard', frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

## Mini Project: Webcam Object Detection with Pre-Trained YOLO

Goal: Use pre-trained YOLO to detect objects through your webcam in real time.

```

from ultralytics import YOLO
import cv2

# Load pre-trained YOLOv8
model = YOLO('yolov8n.pt') # Downloads automatically first time (~6MB)

# Open webcam
cap = cv2.VideoCapture(0)

print("Press 'q' to quit")
print("Try holding up: phone, book, cup, bottle, etc.")

frame_count = 0
detection_log = []

```

```

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1

    # Run YOLO detection
    results = model(frame, verbose=False, conf=0.5) # Only show >50% confidence

    # Log detections
    for result in results:
        for box in result.bboxes:
            name = result.names[int(box.cls)]
            conf = float(box.conf)
            detection_log.append(f"Frame {frame_count}: {name} ({conf:.1%})")

    # Draw results
    annotated = results[0].plot()

    # Add FPS counter
    cv2.putText(annotated, f'Frame: {frame_count}',
                (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

    cv2.imshow('YOLO Webcam Detection', annotated)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

# Print detection summary
print(f"\nProcessed {frame_count} frames")
print(f"Total detections: {len(detection_log)}")
print("\nLast 10 detections:")
for d in detection_log[-10:]:
    print(f" {d}")

```

### What to Try:

- 1. Hold up your phone — YOLO should detect "cell phone"
- 2. Hold up a book — "book"
- 3. Wave a pen — might detect it, might not (small objects are harder)
- 4. Show multiple objects — YOLO detects them all simultaneously
- 5. Move objects around — watch the boxes follow in real time

### What You Will Learn:

- How to use YOLO with just a few lines of code

- Real-time object detection on live video
- How confidence scores work
- How fast YOLO processes frames
- The foundation for ExamGuard's object detection system

### Key Takeaway

YOLO is ExamGuard's eyes for detecting prohibited items. It processes 30+ frames per second, detects multiple objects simultaneously, and can be fine-tuned to recognize exam-specific items like chits and earpieces. Combined with OpenCV for reading camera feeds, YOLO forms the core of ExamGuard's real-time monitoring capability.



# Face Detection and Recognition — Know WHO Is Where

## What Is This?

Face detection and recognition is the ability to:

- 1. Find faces in an image or video (detection)
- 2. Identify whose face it is (recognition)

These are two separate tasks:

Face DETECTION: "There are 30 faces in this exam hall frame"  
(Finds the faces, draws boxes around them)

Face RECOGNITION: "Face at Seat 5 belongs to Student: Ahmed Khan, Roll #2245"  
(Matches the face to a known identity)

You need both for ExamGuard.

## WHY This Matters for ExamGuard

ExamGuard needs to know WHO is sitting WHERE at all times.

### Before the Exam:

Registration Phase:

1. Each student sits at their assigned seat
2. Camera captures their face
3. System stores: "Face #2245 = Ahmed Khan, Seat 5, Hall A"

Database after registration:

Face #2245 → Ahmed Khan, Seat 5  
Face #2246 → Sara Ali, Seat 6  
Face #2247 → Hassan Raza, Seat 7  
... (all 30 students)

### During the Exam:

Real-time monitoring:

Camera frame → Detect all faces → Match each face to database

Seat 5 face matches Ahmed Khan ✓ (correct student in correct seat)

Seat 6 face matches Sara Ali ✓

Seat 7 face does NOT match anyone ✗ → ALERT: Unknown person!

Seat 12 face matches student assigned to Seat 18 ✗ → ALERT: Wrong seat!

### ExamGuard Uses Face Detection For:

| Use Case                | What Happens                                              |
|-------------------------|-----------------------------------------------------------|
| Attendance              | Automatically mark who is present (no manual roll call)   |
| Seat verification       | Confirm each student is in their assigned seat            |
| Impersonation detection | Someone taking exam for another student → caught          |
| Tracking individuals    | When cheating detected, system knows exactly WHO          |
| Evidence                | "Student Ahmed Khan (Roll #2245) was flagged at 10:23 AM" |

## Two Parts Explained

### Part 1: Face Detection — Finding the Face

Face detection locates faces in an image and returns their position (bounding box).

Input image of exam hall:

```
+-----+
|               |
| [face1]  [face2] [face3] |
|               |
| [face4]  [face5] [face6] |
|               |
+-----+
```

Output:

Face 1: position (50, 30) to (100, 90)  
Face 2: position (180, 25) to (230, 85)  
Face 3: position (310, 35) to (360, 95)  
Face 4: position (55, 170) to (105, 230)  
Face 5: position (185, 165) to (235, 225)  
Face 6: position (315, 175) to (365, 235)

### Part 2: Face Recognition — Identifying the Person

Face recognition takes a detected face and compares it to a database of known faces.

Detected face at Seat 5

|  
v

Convert face to 128 numbers (face encoding)

[0.12, -0.34, 0.56, 0.78, ..., -0.23]

|  
v

Compare to all stored face encodings

Ahmed Khan: distance = 0.3 (CLOSE MATCH!)  
Sara Ali: distance = 0.9 (no match)  
Hassan Raza: distance = 0.8 (no match)

|  
v

Result: "This is Ahmed Khan" (distance < 0.6 threshold)

### Libraries for Face Detection/Recognition

| Library          | Best For                     | Speed     | Accuracy | Difficulty |
|------------------|------------------------------|-----------|----------|------------|
| face_recognition | Quick start, simplest API    | Medium    | Good     | Easy       |
| dlib             | Precise landmarks, reliable  | Medium    | Great    | Medium     |
| MediaPipe Face   | Real-time, mobile            | Fast      | Good     | Easy       |
| InsightFace      | Production, highest accuracy | Fast      | Best     | Medium     |
| OpenCV Haar      | Legacy, very fast            | Very fast | OK       | Easy       |

Recommendation: Start with `face\_recognition` library (built on dlib). Move to InsightFace for production.

## Face Detection with face\_recognition Library

```
import face_recognition
import cv2

# Load an image
image = face_recognition.load_image_file('exam_hall.jpg')

# Find all faces in the image
face_locations = face_recognition.face_locations(image)

print(f"Found {len(face_locations)} faces")

# Draw boxes around each face
image_bgr = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
for (top, right, bottom, left) in face_locations:
    cv2.rectangle(image_bgr, (left, top), (right, bottom), (0, 255, 0), 2)

cv2.imshow('Detected Faces', image_bgr)
cv2.waitKey(0)
```

## Face Recognition — Matching to Known People

```
import face_recognition
import cv2
import os
import numpy as np

# ===== Step 1: Register known students =====
known_faces = {} # name → face encoding

# Load each student's photo
student_photos = {
    'Ahmed Khan': 'students/ahmed_khan.jpg',
    'Sara Ali': 'students/sara_ali.jpg',
    'Hassan Raza': 'students/hassan_raza.jpg',
}

for name, photo_path in student_photos.items():
    image = face_recognition.load_image_file(photo_path)
    encoding = face_recognition.face_encodings(image)[0] # 128-number encoding
    known_faces[name] = encoding
    print(f"Registered: {name}")

print(f"\nTotal registered students: {len(known_faces)}")

# ===== Step 2: Identify faces in exam frame =====
exam_frame = face_recognition.load_image_file('exam_frame.jpg')

# Find all faces and their encodings
face_locations = face_recognition.face_locations(exam_frame)
face_encodings = face_recognition.face_encodings(exam_frame, face_locations)
```

```

# Convert for drawing
frame_bgr = cv2.cvtColor(exam_frame, cv2.COLOR_RGB2BGR)

known_names = list(known_faces.keys())
known_encodings = list(known_faces.values())

for (top, right, bottom, left), encoding in zip(face_locations, face_encodings):
    # Compare this face to all known faces
    distances = face_recognition.face_distance(known_encodings, encoding)
    best_match_idx = np.argmin(distances)

    if distances[best_match_idx] < 0.6: # Threshold for match
        name = known_names[best_match_idx]
        confidence = 1 - distances[best_match_idx]
        color = (0, 255, 0) # Green for known student
    else:
        name = "UNKNOWN"
        confidence = 0
        color = (0, 0, 255) # Red for unknown person

    # Draw box and label
    cv2.rectangle(frame_bgr, (left, top), (right, bottom), color, 2)
    cv2.putText(frame_bgr, f'{name} ({confidence:.0%})',
                (left, top - 10), cv2.FONT_HERSHEY_SIMPLEX,
                0.6, color, 2)

cv2.imshow('Face Recognition', frame_bgr)
cv2.waitKey(0)

```

## Face Landmarks — Understanding Facial Features

Face landmarks are specific points on a face (68 or 468 points depending on the model):

Standard 68 landmarks:

- Jaw line: 17 points
- Left eyebrow: 5 points
- Right eyebrow: 5 points
- Nose: 9 points
- Left eye: 6 points
- Right eye: 6 points
- Mouth: 20 points

ExamGuard uses landmarks for:

- Gaze estimation: Eye landmarks show where the student is looking
- Head pose: Which direction the head is turned
- Expression analysis: Nervous/stressed expression patterns

```

import face_recognition
import cv2

```

```

image = face_recognition.load_image_file('student_face.jpg')
landmarks = face_recognition.face_landmarks(image)

```

```

for face in landmarks:
    # face is a dictionary of facial features
    print("Left eye points:", face['left_eye'])
    print("Right eye points:", face['right_eye'])
    print("Nose tip:", face['nose_tip'])

    # Draw landmarks on image
    image_bgr = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    for feature_name, points in face.items():
        for point in points:
            cv2.circle(image_bgr, point, 2, (0, 255, 0), -1)

cv2.imshow('Landmarks', image_bgr)
cv2.waitKey(0)

```

## MediaPipe Face Detection (Faster, Real-Time)

For real-time processing with multiple cameras, MediaPipe is faster:

```

import mediapipe as mp
import cv2

mp_face = mp.solutions.face_detection
mp_draw = mp.solutions.drawing_utils

cap = cv2.VideoCapture(0)

with mp_face.FaceDetection(min_detection_confidence=0.5) as face_detection:
    while True:
        ret, frame = cap.read()
        if not ret:
            break

        # Convert to RGB (MediaPipe needs RGB)
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        # Detect faces
        results = face_detection.process(rgb)

        # Draw detections
        if results.detections:
            for detection in results.detections:
                mp_draw.draw_detection(frame, detection)

                # Get confidence
                confidence = detection.score[0]
                print(f"Face detected: {confidence:.1%} confidence")

cv2.imshow('MediaPipe Face Detection', frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

```

```
cap.release()
cv2.destroyAllWindows()
```

## ExamGuard Face System — Complete Architecture

BEFORE EXAM (Registration):

Student walks in → Camera captures face → Generate encoding → Store in database

Result: Database of 30 face encodings + names + seat assignments

DURING EXAM (Monitoring):

Every 5 seconds:

Camera frame → Detect all faces → Generate encodings → Match to database

For each detected face:

Match found?

YES → Student identified → Check correct seat → All good ✓

NO → ALERT: Unknown person in exam hall!

Face at wrong seat?

→ ALERT: Student in wrong seat (possible impersonation)

Expected face missing for 2+ minutes?

→ ALERT: Student may have left hall

AFTER EXAM (Evidence):

All face detections logged with timestamps

"Ahmed Khan was at Seat 5 from 9:00 AM to 12:00 PM"

"Unknown face detected at Seat 12 at 10:23 AM — evidence clip saved"

## Mini Project: Attendance System

Goal: Build a system that detects who walks in front of the camera and marks them as present.

```
import face_recognition
import cv2
import numpy as np
from datetime import datetime

# ===== Register known people =====
# Take a few photos of yourself and friends/family
# Save in a folder: known_people/name.jpg

import os

known_faces = {}
known_dir = 'known_people'

for filename in os.listdir(known_dir):
    if filename.endswith(('.jpg', '.png')):
        name = os.path.splitext(filename)[0]
        image = face_recognition.load_image_file(os.path.join(known_dir, filename))
        encodings = face_recognition.face_encodings(image)
```

```

    if encodings:
        known_faces[name] = encodings[0]
        print(f"Registered: {name}")

print(f"\nTotal registered: {len(known_faces)} people")

# ===== Run attendance system =====
attendance = {} # name → first seen time
cap = cv2.VideoCapture(0)

known_names = list(known_faces.keys())
known_encodings = list(known_faces.values())

print("\nAttendance system running... Press 'q' to quit and see results.")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Process every 3rd frame for speed
    small_frame = cv2.resize(frame, (0, 0), fx=0.5, fy=0.5)
    rgb_small = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)

    # Detect and encode faces
    face_locations = face_recognition.face_locations(rgb_small)
    face_encodings = face_recognition.face_encodings(rgb_small, face_locations)

    for (top, right, bottom, left), encoding in zip(face_locations, face_encodings):
        # Scale back up (we shrunk the frame by 0.5)
        top *= 2; right *= 2; bottom *= 2; left *= 2

        # Match face
        distances = face_recognition.face_distance(known_encodings, encoding)

        if len(distances) > 0:
            best_idx = np.argmin(distances)
            if distances[best_idx] < 0.6:
                name = known_names[best_idx]
                color = (0, 255, 0)

            # Record attendance
            if name not in attendance:
                attendance[name] = datetime.now().strftime('%H:%M:%S')
                print(f"PRESENT: {name} at {attendance[name]}")
            else:
                name = "Unknown"
                color = (0, 0, 255)
        else:
            name = "Unknown"
            color = (0, 0, 255)

```

```

# Draw
cv2.rectangle(frame, (left, top), (right, bottom), color, 2)
cv2.putText(frame, name, (left, top - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)

# Show attendance count
cv2.putText(frame, f'Present: {len(attendance)}/{len(known_faces)}',
            (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)

cv2.imshow('Attendance System', frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

# ===== Print attendance report =====
print("\n" + "=" * 40)
print("ATTENDANCE REPORT")
print("=" * 40)
for name, time in attendance.items():
    print(f" {name}: Present (arrived {time})")

absent = set(known_faces.keys()) - set(attendance.keys())
for name in absent:
    print(f" {name}: ABSENT")

print(f"\nTotal: {len(attendance)}/{len(known_faces)} present")

```

### What You Will Learn:

- How face encodings work (converting a face to 128 numbers)
- Comparing face encodings to find matches
- Real-time face processing from webcam
- Building a practical face-based system
- The exact architecture ExamGuard uses for student identification

### Connection to ExamGuard:

This attendance system IS ExamGuard's face module. Replace "known\_people" with "registered\_students," add seat assignment checking, and you have ExamGuard's identity verification system.

### Key Takeaway

Face detection and recognition give ExamGuard the ability to answer "WHO is doing WHAT." Without it, ExamGuard can detect cheating behavior but cannot tell you which student was involved. This module connects detected behaviors to specific identities, making evidence actionable and accountability clear.



V

Track keypoints over time (30 frames = 1 second)

|  
v

Analyze movement patterns:

- "Right wrist moved 200px toward neighbor over 2 seconds"
- "Head turned 60 degrees to the left for 8 seconds"
- "Body leaning 30 degrees toward neighbor's desk"

|  
v

Combine signals:

Head turned + Body leaning + Hand extended = CHEATING (92% confidence)

## Models for Pose Estimation

| Model          | Speed     | Accuracy | Best For                    |
|----------------|-----------|----------|-----------------------------|
| MediaPipe Pose | Very fast | Good     | Start here — easiest to use |
| OpenPose       | Slow      | Great    | Multi-person, research      |
| MoveNet        | Fast      | Good     | Real-time, TensorFlow       |
| YOLOv8 Pose    | Fast      | Great    | Already using YOLO          |
| HRNet          | Slow      | Best     | Highest accuracy needed     |

Recommendation: Start with MediaPipe Pose for learning. Use YOLOv8 Pose for ExamGuard production (since you are already using YOLO for object detection).

## The 17 Body Keypoints

Keypoint Map (COCO format):

```
      0: Nose
      /\
    1: L Eye  2: R Eye
    3: L Ear  4: R Ear
      |
    5: L Shoulder---6: R Shoulder
      |       |
    7: L Elbow   8: R Elbow
      |       |
    9: L Wrist  10: R Wrist
      |
   11: L Hip -----12: R Hip
      |       |
   13: L Knee   14: R Knee
      |       |
   15: L Ankle  16: R Ankle
```

For ExamGuard, the most important keypoints are:

- Nose + Eyes (0, 1, 2): Head direction — where are they looking?
- Shoulders (5, 6): Body orientation — are they turned toward a neighbor?
- Wrists (9, 10): Hand position — are they passing something?
- Elbows (7, 8): Arm extension — reaching toward another desk?

## Pose Estimation with MediaPipe

```
import mediapipe as mp
import cv2

# Initialize MediaPipe Pose
mp_pose = mp.solutions.pose
mp_draw = mp.solutions.drawing_utils

cap = cv2.VideoCapture(0)

with mp_pose.Pose(
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
) as pose:

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        # Convert to RGB (MediaPipe needs RGB)
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        # Detect pose
        results = pose.process(rgb)

        # Draw skeleton
        if results.pose_landmarks:
            mp_draw.draw_landmarks(
                frame,
                results.pose_landmarks,
                mp_pose.POSE_CONNECTIONS # Draw lines connecting keypoints
            )

        # Access specific keypoints
        landmarks = results.pose_landmarks.landmark

        nose = landmarks[mp_pose.PoseLandmark.NOSE]
        left_wrist = landmarks[mp_pose.PoseLandmark.LEFT_WRIST]
        right_wrist = landmarks[mp_pose.PoseLandmark.RIGHT_WRIST]
        left_shoulder = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER]
        right_shoulder = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER]

        # Print positions (normalized 0-1)
        h, w = frame.shape[:2]
        print(f"Nose: ({int(nose.x*w)}, {int(nose.y*h)})")
        print(f"L Wrist: ({int(left_wrist.x*w)}, {int(left_wrist.y*h)})")
        print(f"R Wrist: ({int(right_wrist.x*w)}, {int(right_wrist.y*h)})")

cv2.imshow('Pose Estimation', frame)
```

```

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

cap.release()
cv2.destroyAllWindows()

```

## Detecting Suspicious Movements

### Example 1: Hand Reaching Toward Neighbor

```

def check_hand_reaching(landmarks, frame_width):
    """Check if either hand extends past the shoulder line (reaching toward neighbor)."""
    left_shoulder_x = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER].x * frame_width
    right_shoulder_x = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER].x * frame_width
    left_wrist_x = landmarks[mp_pose.PoseLandmark.LEFT_WRIST].x * frame_width
    right_wrist_x = landmarks[mp_pose.PoseLandmark.RIGHT_WRIST].x * frame_width

    shoulder_width = abs(right_shoulder_x - left_shoulder_x)

    # If wrist extends more than 1.5x shoulder width from center → reaching
    center_x = (left_shoulder_x + right_shoulder_x) / 2

    left_reach = abs(left_wrist_x - center_x) > shoulder_width * 1.5
    right_reach = abs(right_wrist_x - center_x) > shoulder_width * 1.5

    if left_reach or right_reach:
        return True, "Hand extending toward neighbor"
    return False, "Normal"

```

### Example 2: Head Turn Detection

```

import math

def check_head_turn(landmarks, frame_width):
    """Check if head is turned significantly (looking at neighbor)."""
    nose_x = landmarks[mp_pose.PoseLandmark.NOSE].x * frame_width
    left_shoulder_x = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER].x * frame_width
    right_shoulder_x = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER].x * frame_width

    # Center of shoulders
    shoulder_center = (left_shoulder_x + right_shoulder_x) / 2
    shoulder_width = abs(right_shoulder_x - left_shoulder_x)

    # How far is nose from shoulder center?
    nose_offset = (nose_x - shoulder_center) / shoulder_width

    if abs(nose_offset) > 0.3: # Head turned more than 30% off-center
        direction = "left" if nose_offset < 0 else "right"
        return True, f"Head turned {direction}"
    return False, "Looking forward"

```

### Example 3: Body Lean Detection

```
def check_body_lean(landmarks, frame_height):
    """Check if body is leaning sideways (toward neighbor's desk)."""
    left_shoulder_y = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER].y * frame_height
    right_shoulder_y = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER].y * frame_height

    shoulder_diff = abs(left_shoulder_y - right_shoulder_y)

    # If shoulders are not level (difference > 5% of frame height)
    if shoulder_diff > frame_height * 0.05:
        direction = "left" if left_shoulder_y > right_shoulder_y else "right"
        return True, f"Body leaning {direction}"
    return False, "Sitting upright"
```

### Tracking Movements Over Time

A single suspicious frame is not cheating. Sustained suspicious behavior IS cheating. You need to track over time.

```
from collections import deque
import time

# Store last 30 frames of analysis (1 second at 30fps)
movement_history = deque(maxlen=30)

def analyze_behavior_over_time(landmarks, frame_size):
    """Analyze sustained suspicious behavior."""
    w, h = frame_size

    # Check current frame
    hand_suspicious, hand_msg = check_hand_reaching(landmarks, w)
    head_suspicious, head_msg = check_head_turn(landmarks, w)
    lean_suspicious, lean_msg = check_body_lean(landmarks, h)

    # Score this frame (0 = normal, 1-3 = suspicious)
    frame_score = sum([hand_suspicious, head_suspicious, lean_suspicious])
    movement_history.append(frame_score)

    # Calculate average suspicion over last second
    if len(movement_history) >= 15: # Need at least 0.5 seconds of data
        avg_score = sum(movement_history) / len(movement_history)

        if avg_score > 2.0:
            return "CHEATING", avg_score, f"{hand_msg}, {head_msg}, {lean_msg}"
        elif avg_score > 1.0:
            return "SUSPICIOUS", avg_score, f"{hand_msg}, {head_msg}, {lean_msg}"

    return "NORMAL", 0, "Normal behavior"
```

## YOLOv8 Pose (Alternative — Recommended for Production)

Since ExamGuard already uses YOLO for object detection, using YOLOv8 Pose means one model handles both.

```
from ultralytics import YOLO
import cv2

# Load YOLOv8 Pose model
model = YOLO('yolov8n-pose.pt')

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Detect poses
    results = model(frame, verbose=False)

    # Access keypoints for each detected person
    for result in results:
        if result.keypoints is not None:
            for person_keypoints in result.keypoints.data:
                # person_keypoints shape: (17, 3) → 17 keypoints, each with (x, y, confidence)
                nose = person_keypoints[0]      # (x, y, conf)
                left_wrist = person_keypoints[9]
                right_wrist = person_keypoints[10]

                print(f"Nose: ({nose[0]:.0f}, {nose[1]:.0f}) conf: {nose[2]:.2f}")

    # Draw results
    annotated = results[0].plot()
    cv2.imshow('YOLOv8 Pose', annotated)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

## Mini Project: Real-Time Pose Detection with Skeleton Overlay

Goal: Detect your body pose from webcam in real time, draw the skeleton, and display which direction you are leaning.

```
import mediapipe as mp
import cv2
import math

mp_pose = mp.solutions.pose
mp_draw = mp.solutions.drawing_utils
```

```

mp_styles = mp.solutions.drawing_styles

cap = cv2.VideoCapture(0)

with mp_pose.Pose(
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
) as pose:

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        h, w = frame.shape[:2]
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = pose.process(rgb)

        status = "No person detected"
        color = (200, 200, 200)

        if results.pose_landmarks:
            # Draw skeleton with styled connections
            mp_draw.draw_landmarks(
                frame,
                results.pose_landmarks,
                mp_pose.POSE_CONNECTIONS,
                mp_styles.get_default_pose_landmarks_style()
            )

            landmarks = results.pose_landmarks.landmark

            # --- Analyze pose ---
            # 1. Head direction
            nose_x = landmarks[mp_pose.PoseLandmark.NOSE].x
            l_shoulder_x = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER].x
            r_shoulder_x = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER].x
            shoulder_center = (l_shoulder_x + r_shoulder_x) / 2
            head_offset = nose_x - shoulder_center

            if head_offset < -0.05:
                head_dir = "Looking LEFT"
            elif head_offset > 0.05:
                head_dir = "Looking RIGHT"
            else:
                head_dir = "Looking CENTER"

            # 2. Body lean
            l_shoulder_y = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER].y
            r_shoulder_y = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER].y
            shoulder_diff = l_shoulder_y - r_shoulder_y

```

```

if shoulder_diff > 0.03:
    lean_dir = "Leaning LEFT"
elif shoulder_diff < -0.03:
    lean_dir = "Leaning RIGHT"
else:
    lean_dir = "Upright"

# 3. Hand position
l_wrist_x = landmarks[mp_pose.PoseLandmark.LEFT_WRIST].x
r_wrist_x = landmarks[mp_pose.PoseLandmark.RIGHT_WRIST].x
shoulder_width = abs(r_shoulder_x - l_shoulder_x)

hand_status = "Hands normal"
if abs(l_wrist_x - shoulder_center) > shoulder_width * 1.5:
    hand_status = "LEFT hand extended!"
if abs(r_wrist_x - shoulder_center) > shoulder_width * 1.5:
    hand_status = "RIGHT hand extended!"

# Display analysis
cv2.putText(frame, f'Head: {head_dir}', (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
cv2.putText(frame, f'Body: {lean_dir}', (10, 60),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
cv2.putText(frame, f'Hands: {hand_status}', (10, 90),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

# Overall assessment
suspicious_count = sum([
    'LEFT' in head_dir or 'RIGHT' in head_dir,
    'Leaning' in lean_dir,
    'extended' in hand_status
])

if suspicious_count >= 2:
    status = "SUSPICIOUS BEHAVIOR"
    color = (0, 0, 255)
elif suspicious_count == 1:
    status = "Slightly unusual"
    color = (0, 165, 255)
else:
    status = "Normal"
    color = (0, 255, 0)

cv2.putText(frame, status, (10, h - 20),
            cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)

cv2.imshow('Pose Analysis', frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

```



```
cap.release()  
cv2.destroyAllWindows()
```

### What to Try:

- 1. Sit normally — should show "Normal"
- 2. Turn your head left/right — should detect head turn
- 3. Lean to one side — should detect body lean
- 4. Extend your arm out — should detect hand extension
- 5. Combine multiple movements — should flag "SUSPICIOUS"

### What You Will Learn:

- Real-time body keypoint detection
- Drawing skeleton overlays on video
- Calculating body angles and positions from keypoints
- Building rule-based behavior analysis from pose data
- The exact logic ExamGuard uses for body language cheating detection

### Key Takeaway

Pose estimation gives ExamGuard the ability to read body language. While face detection tells you WHO the student is and gaze estimation tells you WHERE they are looking, pose estimation tells you WHAT their body is doing. Passing chits, reaching toward neighbors, leaning to copy — all of these are body movements that pose estimation can detect and flag in real time.

## Gaze Estimation — Detecting Where Students Are Looking

### What Is This?

Gaze estimation detects where a person's eyes are looking — their gaze direction. It answers the question: "Is this student looking at their own paper, at their neighbor's paper, at their phone, or at the ceiling?"

What the camera sees:      What gaze estimation tells you:

[student's face]      Gaze direction: 35 degrees left  
Translation: Looking at neighbor's paper  
Duration: 8 seconds  
Status: SUSPICIOUS

This is one of the most powerful signals for cheating detection because eye movement is the hardest thing for students to fake.

### WHY This Is CRITICAL for ExamGuard

Eyes do not lie. A student can keep their body still and their head mostly forward, but if their eyes keep darting to the neighbor's paper, they are copying.

#### The Gaze Rules for Exam Monitoring:

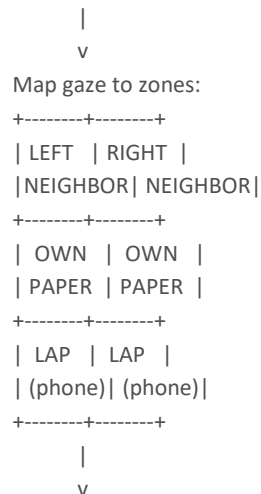
Looking at own paper → NORMAL (expected behavior)  
Looking at question paper → NORMAL (reading questions)  
Looking up (thinking) → NORMAL (recalling information)  
Looking at neighbor briefly → NORMAL (everyone glances sometimes)

Looking at neighbor > 5 sec → SUSPICIOUS (might be copying)  
Looking at lap (phone?) → SUSPICIOUS (hidden device?)  
Repeated glances at neighbor → SUSPICIOUS (pattern of copying)

Looking at neighbor > 5 sec  
+ body leaning toward them → CHEATING (high confidence)  
Looking at lap + hand under desk → CHEATING (phone use)

#### ExamGuard Gaze System:

Camera frame → Detect face → Find eye landmarks → Calculate gaze direction



Track over time:  
"Left neighbor zone: 12 sec in last minute"  
|  
v  
FLAG: Sustained gaze at neighbor

## How Gaze Estimation Works

### Step 1: Detect the Face

Use face detection (from the previous lesson) to find the face in the frame.

### Step 2: Find Eye Landmarks

Locate the specific points around each eye — corners, pupils, iris.

Eye landmarks (6 points per eye):

1---2---3  
|   |  
6---5---4

Iris/Pupil: Center point

### Step 3: Calculate Gaze Direction

Method 1: Pupil position relative to eye corners

Left eye:

+--[ \* ]--+   Pupil centered = looking forward  
+--[\*   ]--+   Pupil left = looking left  
+--[   \*]--+   Pupil right = looking right  
+--[ \* ]--+   Pupil low = looking down  
                 Pupil high = looking up

Method 2: Head pose + eye direction combined

Head angle + Iris position = True gaze direction

### Step 4: Map to Zones

Convert the gaze angle to a meaningful zone (own paper, neighbor, etc.)

### Step 5: Track Over Time

A single glance means nothing. Sustained or repeated gaze at a neighbor = suspicious.

## Models and Libraries for Gaze Estimation

| Tool                | Description                                      | Best For                    |
|---------------------|--------------------------------------------------|-----------------------------|
| MediaPipe Face Mesh | 468 face landmarks including detailed eye points | Start here — easiest        |
| GazeML              | Research model for precise gaze                  | High accuracy needed        |
| L2CS-Net            | Appearance-based gaze estimation                 | Works with any camera angle |
| OpenFace            | Academic tool, very accurate                     | Research                    |
| RT-Gene             | Real-time gaze estimation                        | Production systems          |

Recommendation: Start with MediaPipe Face Mesh for eye landmarks. It is free, fast, and gives you 468 face points including detailed iris tracking.

## Gaze Estimation with MediaPipe

```
import mediapipe as mp
import cv2
import numpy as np

mp_face_mesh = mp.solutions.face_mesh
mp_draw = mp.solutions.drawing_utils

# Eye landmark indices in MediaPipe Face Mesh
# Left eye: indices around the left eye
LEFT_EYE = [362, 382, 381, 380, 374, 373, 390, 249, 263, 466, 388, 387, 386, 385, 384, 398]
RIGHT_EYE = [33, 7, 163, 144, 145, 153, 154, 155, 133, 173, 157, 158, 159, 160, 161, 246]

# Iris landmarks
LEFT_IRIS = [474, 475, 476, 477]
RIGHT_IRIS = [469, 470, 471, 472]

cap = cv2.VideoCapture(0)

with mp_face_mesh.FaceMesh(
    max_num_faces=1,
    refine_landmarks=True, # Includes iris landmarks
    min_detection_confidence=0.5
) as face_mesh:

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        h, w = frame.shape[:2]
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = face_mesh.process(rgb)

        if results.multi_face_landmarks:
            face = results.multi_face_landmarks[0]

            # Get iris centers
            left_iris_pts = [face.landmark[i] for i in LEFT_IRIS]
            right_iris_pts = [face.landmark[i] for i in RIGHT_IRIS]

            left_iris_center = (
                int(np.mean([p.x for p in left_iris_pts]) * w),
                int(np.mean([p.y for p in left_iris_pts]) * h)
            )
            right_iris_center = (
                int(np.mean([p.x for p in right_iris_pts]) * w),
                int(np.mean([p.y for p in right_iris_pts]) * h)
            )
```

```

# Get eye corners for reference
left_eye_inner = face.landmark[362]
left_eye_outer = face.landmark[263]
right_eye_inner = face.landmark[133]
right_eye_outer = face.landmark[33]

# Calculate horizontal gaze ratio for left eye
eye_width = abs(left_eye_outer.x - left_eye_inner.x)
if eye_width > 0:
    iris_relative_x = (np.mean([p.x for p in left_iris_pts]) - left_eye_inner.x) / eye_width
else:
    iris_relative_x = 0.5

# Determine gaze direction
if iris_relative_x < 0.35:
    gaze = "Looking RIGHT"
    color = (0, 165, 255)
elif iris_relative_x > 0.65:
    gaze = "Looking LEFT"
    color = (0, 165, 255)
else:
    gaze = "Looking CENTER"
    color = (0, 255, 0)

# Draw iris points
cv2.circle(frame, left_iris_center, 3, (0, 255, 0), -1)
cv2.circle(frame, right_iris_center, 3, (0, 255, 0), -1)

# Display gaze direction
cv2.putText(frame, gaze, (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)
cv2.putText(frame, f'Iris ratio: {iris_relative_x:.2f}', (10, 60),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1)

cv2.imshow('Gaze Estimation', frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

## Adding Vertical Gaze (Up/Down)

```

def get_gaze_direction(face_landmarks, frame_width, frame_height):
    """Calculate both horizontal and vertical gaze direction."""

    # Get iris positions
    left_iris = [face_landmarks.landmark[i] for i in LEFT_IRIS]
    right_iris = [face_landmarks.landmark[i] for i in RIGHT_IRIS]

    # Average iris position
    iris_x = np.mean([p.x for p in left_iris + right_iris])

```

```

iris_y = np.mean([p.y for p in left_iris + right_iris])

# Eye boundary reference points
# Top of eye, bottom of eye for vertical reference
left_eye_top = face_landmarks.landmark[386].y
left_eye_bottom = face_landmarks.landmark[374].y
eye_height = abs(left_eye_bottom - left_eye_top)

# Horizontal reference
left_eye_inner = face_landmarks.landmark[362].x
left_eye_outer = face_landmarks.landmark[263].x
eye_width = abs(left_eye_outer - left_eye_inner)

# Horizontal ratio (0 = looking right, 1 = looking left)
h_ratio = (np.mean([p.x for p in left_iris]) - left_eye_inner) / eye_width if eye_width > 0 else 0.5

# Vertical ratio (0 = looking up, 1 = looking down)
v_ratio = (np.mean([p.y for p in left_iris]) - left_eye_top) / eye_height if eye_height > 0 else 0.5

# Map to zones
if h_ratio < 0.35:
    h_zone = "right"    # Looking right (from their perspective)
elif h_ratio > 0.65:
    h_zone = "left"     # Looking left
else:
    h_zone = "center"

if v_ratio < 0.35:
    v_zone = "up"
elif v_ratio > 0.65:
    v_zone = "down"
else:
    v_zone = "center"

return h_zone, v_zone, h_ratio, v_ratio

```

## ExamGuard Gaze Zones

```

def map_gaze_to_exam_zone(h_zone, v_zone):
    """
    Map gaze direction to meaningful exam zones.

    Zones (from student's perspective sitting at desk):
    UP + CENTER = Looking at board/ceiling (thinking or reading board)
    CENTER + CENTER = Looking at own paper (normal)
    DOWN + CENTER = Looking at own lap (possible phone)
    LEFT + CENTER = Looking at left neighbor (possible copying)
    RIGHT + CENTER = Looking at right neighbor (possible copying)
    DOWN + LEFT = Looking at left neighbor's paper on desk
    DOWN + RIGHT = Looking at right neighbor's paper on desk
    """

```

```

zone_map = {
    ('center', 'center'): ('own_paper', 'NORMAL'),
    ('center', 'down'): ('own_paper', 'NORMAL'),
    ('center', 'up'): ('thinking', 'NORMAL'),
    ('left', 'center'): ('left_neighbor', 'WATCH'),
    ('right', 'center'): ('right_neighbor', 'WATCH'),
    ('left', 'down'): ('left_desk', 'SUSPICIOUS'),
    ('right', 'down'): ('right_desk', 'SUSPICIOUS'),
    ('left', 'up'): ('left_wall', 'NORMAL'),
    ('right', 'up'): ('right_wall', 'NORMAL'),
}

return zone_map.get((h_zone, v_zone), ('unknown', 'WATCH'))

```

## Tracking Gaze Over Time — The Key to Catching Cheaters

A single glance at a neighbor means nothing. Tracking patterns is everything.

```

from collections import deque
import time

```

```

class GazeTracker:
    def __init__(self, window_seconds=60, fps=30):
        self.history = deque(maxlen=window_seconds * fps)
        self.fps = fps

    def add_gaze(self, zone, status):
        self.history.append({
            'zone': zone,
            'status': status,
            'time': time.time()
        })

    def analyze(self):
        """Analyze gaze patterns over the last minute."""
        if len(self.history) < self.fps * 5: # Need at least 5 seconds of data
            return "NORMAL", "Insufficient data"

        # Count time spent in each zone (last 60 seconds)
        zone_counts = {}
        for entry in self.history:
            zone = entry['zone']
            zone_counts[zone] = zone_counts.get(zone, 0) + 1

        total = len(self.history)

        # Calculate percentages
        neighbor_time = (
            zone_counts.get('left_neighbor', 0) +
            zone_counts.get('right_neighbor', 0) +
            zone_counts.get('left_desk', 0) +
            zone_counts.get('right_desk', 0)

```

```

)
neighbor_pct = neighbor_time / total * 100

own_paper_time = zone_counts.get('own_paper', 0)
own_paper_pct = own_paper_time / total * 100

# Decision rules
if neighbor_pct > 30:
    return "CHEATING", f"Looking at neighbor {neighbor_pct:.0f}% of last minute"
elif neighbor_pct > 15:
    return "SUSPICIOUS", f"Looking at neighbor {neighbor_pct:.0f}% of last minute"
elif own_paper_pct < 40:
    return "SUSPICIOUS", f"Looking at own paper only {own_paper_pct:.0f}% of time"
else:
    return "NORMAL", f"Own paper: {own_paper_pct:.0f}%, Neighbor: {neighbor_pct:.0f}%"

# Usage
tracker = GazeTracker()

# In your main loop:
# h_zone, v_zone, _ = get_gaze_direction(face_landmarks, w, h)
# zone, status = map_gaze_to_exam_zone(h_zone, v_zone)
# tracker.add_gaze(zone, status)
# alert_level, reason = tracker.analyze()

```

## Challenges and Limitations

| Challenge         | Description                               | Mitigation                                           |
|-------------------|-------------------------------------------|------------------------------------------------------|
| Camera angle      | Works best with front-facing cameras      | Install cameras at eye level                         |
| Distance          | Accuracy drops beyond 2-3 meters          | Use higher resolution cameras                        |
| Glasses           | Reflections can confuse iris detection    | Use models trained on glasses-wearing faces          |
| Low light         | Pupil detection is harder in dim lighting | Ensure adequate hall lighting                        |
| Fast eye movement | Quick saccades may be missed              | Higher FPS cameras (60fps)                           |
| Multiple students | Need to track each student separately     | Combine with face detection for per-student tracking |

ExamGuard design decision: Place cameras close to students (every 2 rows) and use 60fps where possible for gaze accuracy.

## Mini Project: Simple Gaze Tracker

Goal: Build a real-time gaze tracker that shows "Looking Left / Right / Center" and displays how long you have been looking in each direction.

```

import mediapipe as mp
import cv2
import numpy as np
import time
from collections import defaultdict

mp_face_mesh = mp.solutions.face_mesh

```



```

LEFT_IRIS = [474, 475, 476, 477]
RIGHT_IRIS = [469, 470, 471, 472]

cap = cv2.VideoCapture(0)

# Track time in each direction
gaze_time = defaultdict(float)
last_gaze = None
last_time = time.time()

with mp_face_mesh.FaceMesh(
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5
) as face_mesh:

    print("Gaze Tracker Running — Press 'q' to quit and see stats")

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        h, w = frame.shape[:2]
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = face_mesh.process(rgb)

        current_gaze = "No face"

        if results.multi_face_landmarks:
            face = results.multi_face_landmarks[0]

            # Get iris positions
            left_iris = [face.landmark[i] for i in LEFT_IRIS]

            # Get eye boundary
            left_inner = face.landmark[362].x
            left_outer = face.landmark[263].x
            eye_width = abs(left_outer - left_inner)

            if eye_width > 0:
                iris_x = np.mean([p.x for p in left_iris])
                ratio = (iris_x - left_inner) / eye_width

            # Vertical
            eye_top = face.landmark[386].y
            eye_bottom = face.landmark[374].y
            eye_h = abs(eye_bottom - eye_top)
            if eye_h > 0:
                iris_y = np.mean([p.y for p in left_iris])

```

```

        v_ratio = (iris_y - eye_top) / eye_h
    else:
        v_ratio = 0.5

    # Determine direction
    if ratio < 0.38:
        h_dir = "RIGHT"
    elif ratio > 0.62:
        h_dir = "LEFT"
    else:
        h_dir = "CENTER"

    if v_ratio < 0.38:
        v_dir = "UP"
    elif v_ratio > 0.62:
        v_dir = "DOWN"
    else:
        v_dir = "CENTER"

    current_gaze = f"{h_dir} {v_dir}"

    # Draw iris markers
    for iris_list in [LEFT_IRIS, RIGHT_IRIS]:
        pts = [(int(face.landmark[i].x * w), int(face.landmark[i].y * h))
               for i in iris_list]
        center = (int(np.mean([p[0] for p in pts])),
                  int(np.mean([p[1] for p in pts])))
        cv2.circle(frame, center, 4, (0, 255, 0), -1)

    # Color based on direction
    if h_dir == "CENTER" and v_dir in ["CENTER", "DOWN"]:
        color = (0, 255, 0)    # Green — looking at paper
    elif h_dir != "CENTER":
        color = (0, 0, 255)    # Red — looking sideways
    else:
        color = (0, 165, 255)  # Orange — looking up

    cv2.putText(frame, f'Gaze: {current_gaze}', (10, 30),
                 cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)

    # Track time
    now = time.time()
    if last_gaze:
        gaze_time[last_gaze] += (now - last_time)
    last_gaze = current_gaze
    last_time = now

    # Display time stats
    total_time = sum(gaze_time.values()) or 1
    y_pos = 70
    for direction, seconds in sorted(gaze_time.items(), key=lambda x: -x[1]):

```

```

    pct = seconds / total_time * 100
    text = f'{direction}: {seconds:.1f}s ({pct:.0f}%)'
    cv2.putText(frame, text, (10, y_pos),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
    y_pos += 25

cv2.imshow('Gaze Tracker', frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

# Final report
print("\n" + "=" * 40)
print("GAZE TRACKING REPORT")
print("=" * 40)
total = sum(gaze_time.values())
for direction, seconds in sorted(gaze_time.items(), key=lambda x: -x[1]):
    pct = seconds / total * 100 if total > 0 else 0
    print(f" {direction:>20}: {seconds:6.1f}s ({pct:5.1f}%)")
print(f"\n Total tracked time: {total:.1f}s")

```

### What to Try:

- 1. Look at your screen (center) — should show green "CENTER"
- 2. Look to the left — should show red "LEFT"
- 3. Look to the right — should show red "RIGHT"
- 4. Look up — should show orange "UP"
- 5. Look down at your keyboard — should show "DOWN"
- 6. After a minute, quit and check the time distribution report

### What You Will Learn:

- How iris tracking works with MediaPipe
- Mapping eye positions to gaze directions
- Tracking gaze distribution over time
- The exact logic ExamGuard uses to flag suspicious eye movements

### Key Takeaway

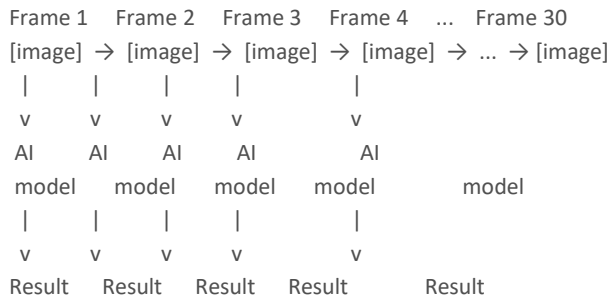
Gaze estimation is one of ExamGuard's most powerful cheating signals. A student can keep their body still and head forward, but their eyes reveal where their attention is. By tracking gaze patterns over time — not just single glances — ExamGuard can distinguish between normal brief eye movements and sustained copying behavior with high confidence. Combined with pose estimation and face detection, gaze estimation completes ExamGuard's ability to understand exactly what each student is doing at every moment.

## Video Processing — Real-Time Frame-by-Frame Analysis

### What Is This?

Video processing means taking a live camera feed or recorded video and analyzing it frame by frame in real time. A video is just a rapid sequence of images (frames) — typically 30 frames per second. Video processing means running your AI models on each of those frames fast enough to keep up.

Video at 30 FPS:



All 30 results must be ready within 1 second, or the system falls behind.

### WHY This Matters for ExamGuard

ExamGuard does not analyze static photos. It analyzes live video streams from exam hall cameras. This creates massive processing demands:

Single camera:

$30 \text{ frames/second} \times 1 \text{ exam hall} = 30 \text{ frames to process per second}$

Real ExamGuard deployment:

$4 \text{ cameras per hall} \times 30 \text{ fps} = 120 \text{ frames/second per hall}$

$10 \text{ halls} \times 120 \text{ fps} = 1,200 \text{ frames/second for full exam}$

$3\text{-hour exam} = 1,200 \times 3600 \times 3 = 12,960,000 \text{ frames total}$

If your processing takes longer than the incoming frame rate, the system falls behind. Frames pile up. By the end of the exam, you might be 30 minutes behind on analysis. That is useless — you need real-time alerts, not alerts after the exam is over.

### Key Concepts

#### FPS (Frames Per Second)

Camera FPS: How many frames the camera captures per second

- Typical: 30 fps
- High quality: 60 fps
- Security cameras: 15-30 fps

Processing FPS: How many frames your AI can process per second

- YOLO on GPU: 30-100+ fps ✓ (can keep up)
- YOLO on CPU: 5-15 fps (falls behind at 30fps input)
- Complex pipeline (YOLO + Pose + Gaze): 10-20 fps

RULE: Processing FPS must be  $\geq$  Camera FPS, or you must skip frames

## Frame Extraction

```
import cv2

cap = cv2.VideoCapture('exam_recording.mp4')

fps = cap.get(cv2.CAP_PROP_FPS)    # Camera FPS
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
duration = total_frames / fps

print(f"Video: {fps} FPS, {total_frames} frames, {duration:.1f} seconds")

frame_count = 0
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame_count += 1

    # Process frame here...

cap.release()
print(f"Processed {frame_count} frames")
```

## Frame Skipping (Critical for Performance)

You do NOT need to process every single frame. If a student is cheating, they will be cheating across many frames. Processing every 3rd frame still catches everything.

```
import cv2

cap = cv2.VideoCapture(0)
frame_count = 0
SKIP_FRAMES = 3 # Process every 3rd frame

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1

    # Skip frames we do not need to process
    if frame_count % SKIP_FRAMES != 0:
        continue

    # Only process every 3rd frame
    # At 30fps, this means processing 10 frames/second — plenty for cheating detection
    process_frame(frame)

cap.release()

ExamGuard math:
```

- Camera: 30 fps
- Skip every 3rd: Process 10 fps
- Cheating event lasts 5+ seconds = 50+ frames analyzed
- More than enough to detect and flag

## Buffering

When processing takes longer than frame arrival, you need a buffer:

Without buffer:

Frame arrives → Process immediately → if slow, DROP frame → miss events

With buffer (queue):

Frame arrives → Add to queue → Process from queue → never miss a frame

If queue grows too large → skip oldest frames (they are stale anyway)

```
import cv2
```

```
import threading
```

```
import queue
```

```
class VideoBuffer:
```

```
    def __init__(self, source, max_buffer=30):
```

```
        self.cap = cv2.VideoCapture(source)
```

```
        self.buffer = queue.Queue(maxsize=max_buffer)
```

```
        self.running = True
```

```
        # Start capture thread
```

```
        self.thread = threading.Thread(target=self._capture, daemon=True)
```

```
        self.thread.start()
```

```
    def _capture(self):
```

```
        """Continuously read frames into buffer."""
```

```
        while self.running:
```

```
            ret, frame = self.cap.read()
```

```
            if not ret:
```

```
                break
```

```
            if self.buffer.full():
```

```
                # Drop oldest frame (it is stale)
```

```
                try:
```

```
                    self.buffer.get_nowait()
```

```
                except queue.Empty:
```

```
                    pass
```

```
            self.buffer.put(frame)
```

```
    def get_frame(self):
```

```
        """Get the next frame to process."""
```

```
        try:
```

```
            return self.buffer.get(timeout=1)
```

```
        except queue.Empty:
```

```
            return None
```

```
def stop(self):
    self.running = False
    self.cap.release()
```

## Multi-Threading for Speed

The biggest performance trick: read frames on one thread, process on another.

Without threading:

Read frame (5ms) → Process frame (50ms) → Read frame (5ms) → Process (50ms)  
Total per frame: 55ms = 18 fps MAX

With threading:

Thread 1 (Reader): Read → Read → Read → Read → Read  
Thread 2 (Processor): Process → Process → Process → Process  
Total: Reader and processor work simultaneously = higher throughput

```
import cv2
import threading
import queue
import time
```

```
class ThreadedVideoProcessor:
```

```
    def __init__(self, source):
        self.cap = cv2.VideoCapture(source)
        self.frame_queue = queue.Queue(maxsize=60)
        self.result_queue = queue.Queue(maxsize=60)
        self.running = True
        self.frames_read = 0
        self.frames_processed = 0
```

```
    def reader_thread(self):
        """Continuously read frames from camera."""
        while self.running:
            ret, frame = self.cap.read()
            if not ret:
                break

            self.frames_read += 1

            if not self.frame_queue.full():
                self.frame_queue.put(frame)
            # If full, skip this frame (drop oldest if needed)
```

```
    def processor_thread(self, process_fn):
        """Continuously process frames from queue."""
        while self.running:
            try:
                frame = self.frame_queue.get(timeout=1)
            except queue.Empty:
                continue
```

```

        # Run AI model on frame
        result = process_fn(frame)
        self.frames_processed += 1

    if not self.result_queue.full():
        self.result_queue.put((frame, result))

def start(self, process_fn):
    """Start reading and processing in separate threads."""
    reader = threading.Thread(target=self.reader_thread, daemon=True)
    processor = threading.Thread(target=self.processor_thread,
                                args=(process_fn,), daemon=True)

    reader.start()
    processor.start()

    return reader, processor

def stop(self):
    self.running = False
    self.cap.release()

# Usage:
def my_ai_model(frame):
    """Your YOLO/CNN/Pose model goes here."""
    # result = model(frame)
    time.sleep(0.03) # Simulate 30ms processing time
    return {"cheating": False, "confidence": 0.1}

processor = ThreadedVideoProcessor(0)
reader, proc = processor.start(my_ai_model)

start_time = time.time()

while True:
    try:
        frame, result = processor.result_queue.get(timeout=1)
    except queue.Empty:
        continue

    # Display result
    elapsed = time.time() - start_time
    fps = processor.frames_processed / elapsed if elapsed > 0 else 0

    cv2.putText(frame, f'Processing FPS: {fps:.1f}', (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
    cv2.putText(frame, f'Read: {processor.frames_read} | Processed: {processor.frames_processed}',
                (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1)

    cv2.imshow('Threaded Processing', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):

```



```
        break

processor.stop()
cv2.destroyAllWindows()
```

## ExamGuard Video Processing Pipeline

```
import cv2
import time
import threading
import queue
from ultralytics import YOLO
from datetime import datetime

class ExamGuardProcessor:
    def __init__(self, camera_url, hall_name, model_path='yolov8n.pt'):
        self.camera_url = camera_url
        self.hall_name = hall_name
        self.model = YOLO(model_path)
        self.cap = cv2.VideoCapture(camera_url)
        self.alerts = []
        self.frame_count = 0
        self.process_every_n = 3 # Process every 3rd frame

    def process_stream(self):
        """Main processing loop for one camera."""
        print(f"[{self.hall_name}] Starting video processing...")

        fps_start = time.time()
        fps_frames = 0

        while True:
            ret, frame = self.cap.read()
            if not ret:
                print(f"[{self.hall_name}] Camera disconnected!")
                break

            self.frame_count += 1

            # Skip frames for performance
            if self.frame_count % self.process_every_n != 0:
                continue

            # Run detection
            results = self.model(frame, verbose=False, conf=0.5)
            fps_frames += 1

            # Check for suspicious objects
            for result in results:
                for box in result.boxes:
                    class_name = result.names[int(box.cls)]
                    confidence = float(box.conf)
```

```

if class_name == 'cell phone' and confidence > 0.7:
    alert = {
        'hall': self.hall_name,
        'object': class_name,
        'confidence': confidence,
        'time': datetime.now().isoformat(),
        'frame': self.frame_count
    }
    self.alerts.append(alert)
    print(f" ALERT: {alert}")

    # Save evidence frame
    x1, y1, x2, y2 = map(int, box.xyxy[0])
    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 0, 255), 3)
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
    cv2.imwrite(f'evidence/{self.hall_name}_{timestamp}.jpg', frame)

# Calculate processing FPS
elapsed = time.time() - fps_start
if elapsed > 0:
    current_fps = fps_frames / elapsed
else:
    current_fps = 0

# Display
cv2.putText(frame, f'{self.hall_name} | FPS: {current_fps:.1f} | Alerts: {len(self.alerts)}',
            (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

self.cap.release()

def stop(self):
    self.cap.release()

# Usage for a single camera:
processor = ExamGuardProcessor(0, 'Hall-A-Cam1')
processor.process_stream()

```

## Performance Optimization Tips

| Technique                                  | Impact        | Effort |
|--------------------------------------------|---------------|--------|
| Frame skipping (every 3rd)                 | 3x faster     | Easy   |
| Resize frames before AI                    | 2-4x faster   | Easy   |
| Use GPU (CUDA)                             | 5-10x faster  | Medium |
| Multi-threading                            | 1.5-2x faster | Medium |
| Batch processing (multiple frames at once) | 2-3x faster   | Medium |
| Use smaller YOLO model (nano vs large)     | 3-5x faster   | Easy   |
| Process ROI only (crop to desk area)       | 2x faster     | Medium |

```
# Quick win: Resize before processing
frame_small = cv2.resize(frame, (640, 480)) # Instead of full 1920x1080
results = model(frame_small)                # 4x fewer pixels to process
```

## Mini Project: Real-Time YOLO on Webcam with FPS Counter

Goal: Read webcam, apply YOLO on each frame, display results with real-time FPS measurement.

```
import cv2
from ultralytics import YOLO
import time

model = YOLO('yolov8n.pt')
cap = cv2.VideoCapture(0)

# FPS tracking
frame_count = 0
start_time = time.time()
fps_display = 0

# Detection tracking
detection_counts = {}

print("Real-time YOLO Detection — Press 'q' to quit")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1

    # Measure processing time for this frame
    frame_start = time.time()

    # Run YOLO
    results = model(frame, verbose=False, conf=0.4)

    frame_time = time.time() - frame_start

    # Count detections
    for result in results:
        for box in result.boxes:
            name = result.names[int(box.cls)]
            detection_counts[name] = detection_counts.get(name, 0) + 1

    # Draw detections
    annotated = results[0].plot()

    # Calculate FPS (update every 10 frames for stability)
    if frame_count % 10 == 0:
        elapsed = time.time() - start_time
```

```

fps_display = frame_count / elapsed

# Display stats
cv2.putText(annotated, f'FPS: {fps_display:.1f}', (10, 30),
             cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
cv2.putText(annotated, f'Frame time: {frame_time*1000:.0f}ms', (10, 60),
             cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1)
cv2.putText(annotated, f'Total frames: {frame_count}', (10, 85),
             cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1)

# Show detection counts
y = 115
for name, count in sorted(detection_counts.items(), key=lambda x: -x[1][:5]):
    cv2.putText(annotated, f'{name}: {count}', (10, y),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (200, 200, 200), 1)
    y += 20

cv2.imshow('Real-Time YOLO', annotated)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

# Summary
elapsed = time.time() - start_time
print(f"\nSession Summary:")
print(f" Duration: {elapsed:.1f} seconds")
print(f" Frames processed: {frame_count}")
print(f" Average FPS: {frame_count/elapsed:.1f}")
print(f"\nDetection counts:")
for name, count in sorted(detection_counts.items(), key=lambda x: -x[1]):
    print(f" {name}: {count}")

```

### What You Will Learn:

- Real-time video processing with AI models
- Measuring and displaying FPS
- Understanding processing bottlenecks
- How fast YOLO actually runs on your hardware
- The complete frame-to-detection pipeline ExamGuard uses

### Key Takeaway

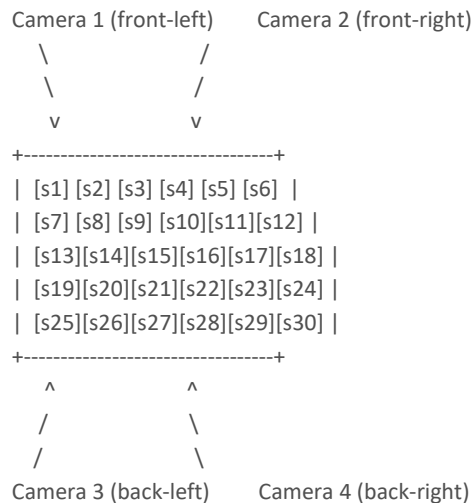
Video processing is the bridge between AI models and real-world deployment. Your models might work great on single images, but ExamGuard needs them to work on continuous video streams at 30+ FPS across multiple cameras. The techniques in this lesson — frame skipping, multi-threading, buffering, and performance measurement — are what make the difference between a demo and a production system.

## Multi-Camera Systems — Handling 4+ Cameras Per Hall

### What Is This?

Multi-camera processing means reading, processing, and coordinating video feeds from multiple cameras simultaneously. A single camera gives you one angle. Multiple cameras give you complete coverage — no blind spots, multiple perspectives on the same student.

Exam Hall A:



4 cameras  $\times$  30 FPS = 120 frames/second to process

### WHY ExamGuard Needs Multiple Cameras

#### Single Camera Problems:

Problem 1: Blind spots

Camera at front  $\rightarrow$  Cannot see students in back row clearly

Camera at back  $\rightarrow$  Cannot see front students' desk surfaces

Problem 2: Occlusion

Student in row 2 blocks view of student in row 3

Cannot see hands under desks from any single angle

Problem 3: Angle limitations

Front camera  $\rightarrow$  Good for gaze detection (seeing eyes)

Side camera  $\rightarrow$  Good for body pose (seeing hands reaching)

One angle cannot capture everything

#### Multi-Camera Solution:

Camera 1 (front-left): Covers rows 1-3, good for faces/gaze

Camera 2 (front-right): Covers rows 1-3, different angle

Camera 3 (back-left): Covers rows 3-5, sees desk surfaces

Camera 4 (back-right): Covers rows 3-5, catches passing items

Combined: Full coverage, multiple angles, no blind spots

### ExamGuard Scale:

| Deployment                         | Cameras | FPS Each | Total FPS |
|------------------------------------|---------|----------|-----------|
| 1 hall (30 students)               | 4       | 30       | 120       |
| 5 halls (150 students)             | 20      | 30       | 600       |
| Full exam (10 halls, 300 students) | 40      | 30       | 1,200     |
| University-wide (50 halls)         | 200     | 30       | 6,000     |

## RTSP Streams — How Cameras Send Video Over Network

Security cameras and IP cameras use RTSP (Real-Time Streaming Protocol) to send video over the network. Instead of USB cables, cameras stream over WiFi or Ethernet.

IP Camera → Network (WiFi/Ethernet) → RTSP URL → Your Python code

RTSP URL format:

```
rtsp://username:password@camera_ip:port/stream_path
```

Examples:

```
rtsp://admin:password@192.168.1.100:554/stream1
```

```
rtsp://admin:admin123@192.168.1.101:554/cam/realmonitor?channel=1
```

```
import cv2
```

```
# Connect to an IP camera via RTSP
```

```
cap = cv2.VideoCapture('rtsp://admin:password@192.168.1.100:554/stream1')
```

```
if not cap.isOpened():
```

```
    print("ERROR: Cannot connect to camera!")
```

```
else:
```

```
    print("Connected to camera successfully")
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        print("Lost connection!")
```

```
        break
```

```
    cv2.imshow('IP Camera', frame)
```

```
    if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
        break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

## Multi-Threaded Camera Reading

The most important architecture decision: each camera gets its own thread.

If you read cameras one by one (sequentially), you can only process one camera at a time:

Sequential (WRONG):

```
Read Cam1 → Process Cam1 → Read Cam2 → Process Cam2 → Read Cam3 → ...
```

Time: 4 cameras × 50ms each = 200ms per cycle = 5 FPS (terrible)

Multi-threaded (CORRECT):

Thread 1: Read Cam1 → Process Cam1 → Read Cam1 → ...

Thread 2: Read Cam2 → Process Cam2 → Read Cam2 → ...

Thread 3: Read Cam3 → Process Cam3 → Read Cam3 → ...

Thread 4: Read Cam4 → Process Cam4 → Read Cam4 → ...

Time: All cameras process simultaneously = 20+ FPS each

```
import cv2
```

```
import threading
```

```
import queue
```

```
import time
```

```
class CameraStream:
```

```
    """Handles a single camera in its own thread."""
```

```
    def __init__(self, source, name, max_queue=30):
```

```
        self.source = source
```

```
        self.name = name
```

```
        self.cap = cv2.VideoCapture(source)
```

```
        self.frame_queue = queue.Queue(maxsize=max_queue)
```

```
        self.running = True
```

```
        self.frames_read = 0
```

```
        self.connected = self.cap.isOpened()
```

```
        if self.connected:
```

```
            self.thread = threading.Thread(target=self._read_loop, daemon=True)
```

```
            self.thread.start()
```

```
            print(f"[{name}] Connected and streaming")
```

```
        else:
```

```
            print(f"[{name}] FAILED to connect to {source}")
```

```
    def _read_loop(self):
```

```
        """Continuously read frames in background thread."""
```

```
        while self.running:
```

```
            ret, frame = self.cap.read()
```

```
            if not ret:
```

```
                print(f"[{self.name}] Lost connection!")
```

```
                self.connected = False
```

```
                break
```

```
            self.frames_read += 1
```

```
            # Drop old frames if queue is full (always keep latest)
```

```
            if self.frame_queue.full():
```

```
                try:
```

```
                    self.frame_queue.get_nowait()
```

```
                except queue.Empty:
```

```
                    pass
```

```
            self.frame_queue.put(frame)
```

```
    def get_frame(self):
```

```

        """Get the latest frame (non-blocking)."""
        try:
            return self.frame_queue.get_nowait()
        except queue.Empty:
            return None

    def stop(self):
        self.running = False
        self.cap.release()

class MultiCameraManager:
    """Manages multiple camera streams."""

    def __init__(self):
        self.cameras = {}

    def add_camera(self, source, name):
        """Add a camera stream."""
        cam = CameraStream(source, name)
        self.cameras[name] = cam
        return cam

    def get_all_frames(self):
        """Get latest frame from each camera."""
        frames = {}
        for name, cam in self.cameras.items():
            frame = cam.get_frame()
            if frame is not None:
                frames[name] = frame
        return frames

    def stop_all(self):
        """Stop all camera streams."""
        for cam in self.cameras.values():
            cam.stop()

    def status(self):
        """Print status of all cameras."""
        for name, cam in self.cameras.items():
            status = "CONNECTED" if cam.connected else "DISCONNECTED"
            print(f" [{name}] {status} | Frames read: {cam.frames_read}")

```

## Using Multi-Camera Manager

```

import cv2
import numpy as np

# Create manager
manager = MultiCameraManager()

# Add cameras (use webcam indices for testing, RTSP URLs for real cameras)

```



```

manager.add_camera(0, 'Cam-Front-Left')
# manager.add_camera(1, 'Cam-Front-Right')      # Second USB camera
# manager.add_camera('rtsp://...', 'Cam-Back-Left') # IP camera
# manager.add_camera('rtsp://...', 'Cam-Back-Right') # IP camera

# For testing with single webcam, use video files:
# manager.add_camera('video1.mp4', 'Cam-Front-Left')
# manager.add_camera('video2.mp4', 'Cam-Back-Right')

time.sleep(1) # Give cameras time to initialize

while True:
    frames = manager.get_all_frames()

    if not frames:
        continue

    # Display all camera feeds in a grid
    display_frames = []
    for name, frame in frames.items():
        # Resize for display
        small = cv2.resize(frame, (640, 480))
        cv2.putText(small, name, (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
        display_frames.append(small)

    # Arrange in 2x2 grid
    if len(display_frames) >= 2:
        row1 = np.hstack(display_frames[:2])
        if len(display_frames) >= 4:
            row2 = np.hstack(display_frames[2:4])
            grid = np.vstack([row1, row2])
        else:
            grid = row1
    elif len(display_frames) == 1:
        grid = display_frames[0]
    else:
        continue

    cv2.imshow('ExamGuard Multi-Camera', grid)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

manager.stop_all()
cv2.destroyAllWindows()

```

## Person Re-Identification (Re-ID)

The Challenge: Student appears in Camera 1, then walks and appears in Camera 3. How does ExamGuard know it is the same person?

Camera 1 frame: [person detected, bounding box, appearance features]

Camera 3 frame: [person detected, bounding box, appearance features]

Re-ID model compares appearance features:

Camera 1 person features: [0.23, 0.87, -0.12, ...] (128 numbers)

Camera 3 person features: [0.25, 0.85, -0.10, ...] (128 numbers)

Distance: 0.15 → SAME PERSON (below threshold)

ExamGuard uses Re-ID for:

- Tracking a student who leaves their seat and moves around
- Ensuring no one enters from another hall
- Matching suspicious person across multiple camera angles

# Simplified Re-ID concept using appearance features

```
import torch
```

```
import torchvision.models as models
```

```
import torchvision.transforms as transforms
```

```
from PIL import Image
```

```
import numpy as np
```

# Use a pre-trained model as feature extractor

```
model = models.resnet50(pretrained=True)
```

```
model = torch.nn.Sequential(*list(model.children())[:-1]) # Remove last layer
```

```
model.eval()
```

```
transform = transforms.Compose([
    transforms.Resize((256, 128)), # Standard Re-ID size
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

```
def extract_features(person_crop):
    """Extract appearance features from a person crop."""
    img = Image.fromarray(person_crop)
    img_tensor = transform(img).unsqueeze(0)
```

```
    with torch.no_grad():
        features = model(img_tensor)
```

```
    return features.squeeze().numpy()
```

```
def match_person(features1, features2, threshold=0.5):
    """Check if two person crops are the same person."""
    distance = np.linalg.norm(features1 - features2)
    is_same = distance < threshold
    return is_same, distance
```

## Edge Processing Architecture

For large deployments (50+ cameras), you cannot send all raw video to one central server. Instead, use edge processing — each camera has a small computer that pre-processes locally.

WITHOUT edge processing (BAD for scale):

Camera 1 —(raw video)—→  
Camera 2 —(raw video)—→ Central Server (overwhelmed)  
Camera 3 —(raw video)—→  
...  
Camera 50 —(raw video)—→

Problem: 50 cameras × 30fps × 2MB/frame = 3 GB/second network traffic!

WITH edge processing (GOOD for scale):

Camera 1 → [Edge Device] → (only alerts + suspicious clips) →  
Camera 2 → [Edge Device] → (only alerts + suspicious clips) → Central Server  
Camera 3 → [Edge Device] → (only alerts + suspicious clips) → (handles easily)  
...  
Camera 50 → [Edge Device] → (only alerts + suspicious clips)→

Each edge device runs YOLO locally

Only sends data when something suspicious is detected

Network traffic reduced by 95%+

**Edge devices: Raspberry Pi, NVIDIA Jetson Nano, or any small computer with GPU capability.**

# Edge device code (runs on small computer attached to each camera)

```
class EdgeProcessor:
    def __init__(self, camera_url, server_url, hall_name, camera_name):
        self.cap = cv2.VideoCapture(camera_url)
        self.model = YOLO('yolov8n.pt') # Lightweight model for edge
        self.server_url = server_url
        self.hall_name = hall_name
        self.camera_name = camera_name

    def process_and_send(self):
        """Process locally, only send suspicious findings."""
        while True:
            ret, frame = self.cap.read()
            if not ret:
                break

            results = self.model(frame, verbose=False, conf=0.5)

            suspicious = False
            for result in results:
                for box in result.bboxes:
                    class_name = result.names[int(box.cls)]
                    if class_name in ['cell phone', 'book']: # Suspicious objects
                        suspicious = True
                        break

            if suspicious:
                # Only NOW send data to server
                self.send_alert(frame, results)
```

```

        print(f"[{self.camera_name}] Alert sent to server")

    # Normal frames → do nothing (save bandwidth)

def send_alert(self, frame, results):
    """Send suspicious frame and detections to central server."""
    # In real system: HTTP POST or WebSocket to server
    # For now, save locally
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
    cv2.imwrite(f'alerts/{self.hall_name}_{self.camera_name}_{timestamp}.jpg', frame)

```

## Synchronizing Multiple Camera Feeds

When combining data from multiple cameras, timestamps must be synchronized:

```

import time
from datetime import datetime

class SynchronizedMultiCamera:
    def __init__(self):
        self.cameras = {}
        self.sync_buffer = {} # timestamp → {camera_name: frame}

    def add_camera(self, source, name):
        self.cameras[name] = CameraStream(source, name)

    def get_synchronized_frames(self, max_time_diff_ms=100):
        """Get frames from all cameras captured within 100ms of each other."""
        current_time = time.time()
        frames = {}

        for name, cam in self.cameras.items():
            frame = cam.get_frame()
            if frame is not None:
                frames[name] = {
                    'frame': frame,
                    'timestamp': current_time
                }

        # Check if all frames are within acceptable time window
        if len(frames) == len(self.cameras):
            timestamps = [f['timestamp'] for f in frames.values()]
            time_spread = (max(timestamps) - min(timestamps)) * 1000 # ms

            if time_spread <= max_time_diff_ms:
                return {name: f['frame'] for name, f in frames.items()}

        return None # Frames not synchronized

```

## Mini Project: Read 2 Camera Feeds Simultaneously

Goal: Read from two video sources at the same time, detect people in both, and display in a side-by-side view.

```

import cv2
import numpy as np
import threading
import queue
import time
from ultralytics import YOLO

class SimpleCamera:
    """Simple threaded camera reader."""
    def __init__(self, source, name):
        self.name = name
        self.cap = cv2.VideoCapture(source)
        self.latest_frame = None
        self.lock = threading.Lock()
        self.running = True

        self.thread = threading.Thread(target=self._read, daemon=True)
        self.thread.start()
        print(f"[{name}] Started")

    def _read(self):
        while self.running:
            ret, frame = self.cap.read()
            if not ret:
                break
            with self.lock:
                self.latest_frame = frame

    def get_frame(self):
        with self.lock:
            return self.latest_frame.copy() if self.latest_frame is not None else None

    def stop(self):
        self.running = False
        self.cap.release()

# Initialize YOLO
model = YOLO('yolov8n.pt')

# Start two camera streams
# For testing: use webcam (0) and a video file
# For real use: use two webcams (0, 1) or RTSP URLs
cam1 = SimpleCamera(0, 'Camera-1')
cam2 = SimpleCamera('test_video.mp4', 'Camera-2') # Replace with second source

# If you only have one webcam, use the same source twice for testing:
# cam2 = SimpleCamera(0, 'Camera-2')

time.sleep(1) # Let cameras initialize

```

```

person_count = {'Camera-1': 0, 'Camera-2': 0}

print("\nDual Camera Detection Running — Press 'q' to quit")

while True:
    frame1 = cam1.get_frame()
    frame2 = cam2.get_frame()

    if frame1 is None and frame2 is None:
        continue

    display_frames = []

    for cam_name, frame in [('Camera-1', frame1), ('Camera-2', frame2)]:
        if frame is None:
            # Create placeholder for missing camera
            placeholder = np.zeros((480, 640, 3), dtype=np.uint8)
            cv2.putText(placeholder, f'{cam_name}: No Signal', (50, 240),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
            display_frames.append(placeholder)
            continue

        # Resize for processing and display
        frame = cv2.resize(frame, (640, 480))

        # Run YOLO detection
        results = model(frame, verbose=False, conf=0.5, classes=[0]) # class 0 = person

        # Count people
        people_in_frame = 0
        for result in results:
            for box in result.bboxes:
                people_in_frame += 1
                x1, y1, x2, y2 = map(int, box.xyxy[0])
                conf = float(box.conf)
                cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
                cv2.putText(frame, f'Person {conf:.0%}', (x1, y1-10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

        person_count[cam_name] = people_in_frame

        # Add camera label
        cv2.putText(frame, f'{cam_name} | People: {people_in_frame}',
                    (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)

        display_frames.append(frame)

    # Side by side display
    if len(display_frames) == 2:
        combined = np.hstack(display_frames)
    else:

```

```

combined = display_frames[0]

# Total count across all cameras
total = sum(person_count.values())
cv2.putText(combined, f'Total people across cameras: {total}',
            (10, combined.shape[0] - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

cv2.imshow('Multi-Camera Detection', combined)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cam1.stop()
cam2.stop()
cv2.destroyAllWindows()

print(f"\nFinal person counts:")
for cam, count in person_count.items():
    print(f" {cam}: {count} people")

```

### What You Will Learn:

- Reading multiple video sources simultaneously using threads
- Processing each camera feed independently
- Combining multiple views into one display
- The exact architecture ExamGuard uses for multi-camera monitoring
- Thread safety with locks when sharing data between threads

### Scaling Up:

After this works with 2 cameras, scaling to 4 is just adding more `SimpleCamera` instances. The architecture is the same.

### Key Takeaway

Multi-camera handling is what separates a demo from a real ExamGuard deployment. A single camera gives you a proof of concept. Four cameras per hall with synchronized processing, edge computing, and person re-identification gives you a production surveillance system. The threading and queue patterns you learn here are used in every real-time multi-camera system in the world.

## Phase 5: Computer Vision — Learning Resources

### YouTube Tutorials (Free)

#### OpenCV

- "OpenCV Course — Full Tutorial with Python" by freeCodeCamp
- 4 hours, covers everything from basics to projects
- <https://www.youtube.com/watch?v=oXlwWbU8l2o>
- "OpenCV Python Tutorial For Beginners" by ProgrammingKnowledge
- 40+ video playlist, very beginner friendly
- "Learn OpenCV in 3 Hours" by Murtaza's Workshop
- Fast-paced but practical, builds real projects

#### YOLO Object Detection

- "YOLO Object Detection Tutorial" by Ultralytics (official)
- YOLOv8 official tutorials and documentation
- <https://docs.ultralytics.com/>
- "YOLOv8 Custom Object Detection" — search for latest tutorials
- Many creators show how to train YOLO on custom datasets
- "Object Detection in 5 Minutes" by Fireship
- Quick overview of how object detection works
- "Train YOLOv8 on Custom Dataset" by Roboflow
- End-to-end tutorial with data labeling

#### Face Detection and Recognition

- "Face Recognition with Python" by sentdex
- Uses the face\_recognition library (same as our lessons)
- "Build a Face Recognition System" by Tech With Tim
- Complete project tutorial
- "MediaPipe Face Mesh Tutorial" — search for latest
- 468 face landmarks including iris tracking

#### Pose Estimation

- "MediaPipe Pose Estimation Tutorial" by Nicholas Renotte
- Excellent hands-on tutorial
- "Human Pose Estimation Explained" by Computer Vision Engineer
- Visual explanation of how keypoints work
- "OpenPose vs MediaPipe vs MoveNet" — comparison tutorials
- Helps you understand which to use when

#### Gaze Estimation

- "Eye Tracking with Python" — search YouTube
- Several tutorials using MediaPipe for iris tracking



- "Gaze Estimation with OpenCV and dlib" — search YouTube
- Lower-level approach, good for understanding
- "MediaPipe Iris — Real-time Eye Tracking" by Ivan Googol
- Practical implementation

### Real-Time Video Processing

- "Real-time Object Detection with YOLO and Webcam" — many tutorials available
- This is the most popular demo project on YouTube
- "Multi-threading in Python for Computer Vision" — search YouTube
- Critical for multi-camera processing
- "RTSP Camera Stream with Python" — search YouTube
- How to connect to IP cameras

## Courses (Structured Learning)

### Free

- "OpenCV Python Tutorial" by freeCodeCamp (YouTube)
- Best free OpenCV course available
- Covers reading images, video, drawing, transforms, and projects
- "Computer Vision with Python" on Kaggle Learn
- <https://www.kaggle.com/learn/computer-vision>
- Free micro-course, hands-on in browser
- "Introduction to Computer Vision" by Udacity
- Georgia Tech course available free on Udacity
- More academic, covers theory and math

### Paid (Worth It)

- "Deep Learning and Computer Vision A-Z" on Udemy
- Covers OpenCV, YOLO, face detection, and more
- Frequently on sale for \$10-15
- "Modern Computer Vision with PyTorch" by V Kishore Ayyadevara
- Book + code, very practical

## Practice: Build Each Mini Project in Order

This is the recommended progression. Each project builds on the previous one:

Week 1: OpenCV Basics

- Mini project: Motion detection in video
- Skills: Read video, process frames, draw on images

Week 2: YOLO Object Detection

- Mini project: Real-time webcam object detection
- Skills: Use pre-trained YOLO, detect objects, draw boxes

Week 3: Face Detection and Recognition

- Mini project: Attendance system with face recognition
- Skills: Detect faces, encode faces, match identities

#### Week 4: Pose Estimation

- Mini project: Real-time skeleton overlay from webcam
- Skills: Detect body keypoints, analyze body position

#### Week 5: Gaze Estimation

- Mini project: Gaze direction tracker (left/right/center)
- Skills: Eye landmark detection, iris tracking, gaze mapping

#### Week 6: Video Processing

- Mini project: Real-time YOLO with FPS measurement
- Skills: Frame skipping, multi-threading, performance optimization

#### Week 7: Multi-Camera

- Mini project: Two camera feeds with simultaneous detection
- Skills: Thread management, synchronized processing, grid display

#### Week 8: Integration

- Combine all projects into a basic ExamGuard prototype
- Skills: Pipeline building, multi-model processing

## GitHub Repositories

### Essential

- [ultralytics/ultralytics](https://github.com/ultralytics/ultralytics) — YOLOv8 (object detection)
- <https://github.com/ultralytics/ultralytics>
- The YOLO implementation you will use for ExamGuard
- [google/mediapipe](https://github.com/google/mediapipe) — Pose estimation, face mesh, iris tracking
- <https://github.com/google/mediapipe>
- Multi-platform, works on Python, mobile, and web
- [ageitgey/face\\_recognition](https://github.com/ageitgey/face_recognition) — Simplest face recognition library
- [https://github.com/ageitgey/face\\_recognition](https://github.com/ageitgey/face_recognition)
- Built on dlib, extremely easy API

### Advanced (Reference for Later)

- [opencv/opencv](https://github.com/opencv/opencv) — OpenCV source code and examples
- <https://github.com/opencv/opencv>
- [CMU-Perceptual-Computing-Lab/openpose](https://github.com/CMU-Perceptual-Computing-Lab/openpose) — Multi-person pose estimation
- <https://github.com/CMU-Perceptual-Computing-Lab/openpose>
- [deepinsight/insightface](https://github.com/deepinsight/insightface) — Production face recognition
- <https://github.com/deepinsight/insightface>

## Tools for Data Labeling

When you start training custom YOLO models for ExamGuard, you need to label data:

| Tool         | Type        | Best For                              | Link                                                                                          |
|--------------|-------------|---------------------------------------|-----------------------------------------------------------------------------------------------|
| LabelImg     | Desktop app | Simple bounding box labeling          | <a href="https://github.com/HumanSignal/labelimg">https://github.com/HumanSignal/labelimg</a> |
| Roboflow     | Web app     | Easy labeling + augmentation + export | <a href="https://roboflow.com/">https://roboflow.com/</a>                                     |
| CVAT         | Web app     | Large team labeling projects          | <a href="https://www.cvat.ai/">https://www.cvat.ai/</a>                                       |
| Label Studio | Web app     | Multi-type labeling                   | <a href="https://labelstud.io/">https://labelstud.io/</a>                                     |

Recommendation: Start with Roboflow — free tier, easy to use, exports directly to YOLO format.

## Hardware Recommendations

### For Learning (What You Have Now)

- Any laptop with webcam
- CPU is fine for learning — models will be slower but work
- 8GB RAM minimum

### For Development

- Laptop/desktop with NVIDIA GPU (GTX 1060 or better)
- 16GB RAM
- SSD for fast data loading
- External webcam for better quality

### For ExamGuard Deployment

- NVIDIA Jetson Nano/Xavier (edge processing per camera)
- Central server with RTX 3060+ GPU
- IP cameras (Hikvision, Dahua) with RTSP support
- Gigabit network switch

## Quick Reference: What to Search When Stuck

| Problem               | Search This                                        |
|-----------------------|----------------------------------------------------|
| OpenCV installation   | "pip install opencv-python" (just use pip)         |
| Camera not opening    | "opencv videocapture not working [your OS]"        |
| YOLO too slow         | "speed up yolov8 inference" or use smaller model   |
| Face not detected     | "improve face detection accuracy [lighting/angle]" |
| RTSP not connecting   | "opencv rtsp connection timeout fix"               |
| Multi-threading crash | "python threading opencv safe"                     |
| Gaze inaccurate       | "improve gaze estimation accuracy tips"            |
| GPU not detected      | "pytorch cuda not available fix"                   |
| Out of memory         | "reduce opencv memory usage multi-camera"          |

## Key Tip

Phase 5 is the most hands-on phase. Every lesson has a mini project that produces a working demo. Do not skip the projects — they are not optional exercises, they are the building blocks of ExamGuard. By the end of Phase 5, you should be able to combine all these mini projects into a basic prototype that reads a camera, detects objects, recognizes faces, tracks poses, and estimates gaze. That prototype IS ExamGuard v0.1.